

03 · 저장과 화면: 무엇을 어디서 보나

보강 안내 1/4 · 현재 화면, 과거 이력, 서비스 건강 상태

화면	직접 읽는 곳	사람이 하는 일
FUXA	현재 값·제어: Modbus / 최근 분석 알람: MQTT	지금 설비를 보고 기동·정지·설정값 변경
Grafana · 공정	InfluxDB process_raw / process / anomaly / alerts	시간별 추세와 이상 전후를 비교
Grafana · 인프라	Prometheus	Kafka 랙·Flink 체크포인트·컨테이너 상태 확인

Grafana가 없어도 되는 경로

FUXA의 직접 계측·조작과 Flink의 이상 감지는 Grafana 화면을 거치지 않는다. Grafana는 저장된 이력과 서비스 지표를 설명할 때 유용하다.

두 저장소의 경계

InfluxDB

센서의 시간별 값, 이상 점수, 알람 이력.

Prometheus

SCADA를 구성하는 서비스의 실행 지표. 공정 센서의 주 저장소가 아니다.

03 · 조작은 아래로, 관측은 다시 위로

보강 안내 2/4 · 양방향 SCADA를 한 바퀴로 읽기

운전원 → FUXA 브라우저

교반기 정지 같은 허용 조작을 선택한다.



FUXA 서버 → Modbus TCP

해당 코일 또는 설정 레지스터에 쓴다.



가상 설비

명령을 반영하고 물리 모델의 상태를 바꾼다.



FUXA 되읽기

운전 상태가 바뀌었는지 현재 값을 다시 본다.



수집 → Kafka → InfluxDB → Grafana

새 센서값이 이력에도 나타나는지 별도 경로로 본다.

명령 접수와 실제 반응은 다르다

버튼 클릭, Modbus 쓰기 응답, 새 센서값은 서로 다른 관찰점이다. 시뮬레이터 확인을 실물 모터의 독립 계측이나 정비 완료로 확대하지 않는다.

03 · 화면을 보며 질문하기

보강 안내 3/4 · 실제 UI의 어느 구역을 볼까

FUXA 공정 그림

탱크 → 펌프 → 반응기 → 밸브의 흐름과 현재 상태. 수치는 Modbus 읽기 결과.

FUXA 운전원 조작

기동·정지 버튼, 설정값, 인터록 및 ACK. 조작은 가상 설비에 직접 전달.

FUXA 최근 분석 알람

Flink 분석 결과가 Kafka → Telegraf → MQTT를 거쳐 보이는 마지막 이벤트.

Grafana 공정·알람

InfluxDB에 쌓인 시간별 기록. 이상 전후 추세와 알람 이력을 비교.

Grafana 인프라

Prometheus 지표. 파이프라인이 느리거나 멈췄는지 확인.

03 · 세 권을 한 장에 다시 연결

보강 안내 4/4 · 원문은 뒤에서 해당 주제를 깊게 다룸

권	입력	출력 / 사람이 보는 결과
01 현장·수집	가상 설비의 물리 상태	Modbus 숫자 → MQTT → Kafka raw. FUXA 현재값.
02 흐름·탐지	Kafka raw	clean · anomaly.score · alerts. 검출 근거.
03 저장·관제	Kafka raw 및 분석 출력	InfluxDB·Grafana 이력, MQTT·FUXA 최근 알람.

읽기 순서

먼저 새 보강 4쪽에서 길을 잡고, 뒤의 기존 본문에서 각 기술의 원리·설정·수치를 찾아보면 된다. 기존 본문의 성능값은 해당 기록 시점의 측정이다.

근거: fuxa/build_project.py, telegraf/sink.conf, alert-republish.conf, grafana/provisioning/datasources/datasources.yml 및 기존 검수 기록. 세 권은 IoT-SCADA 범위에 집중한다.

03

저장·관제·전체 동작

이력과 화면, 조작과 검증을 하나의 고리로 연결하기

중복 설명을 통합한 3 권 읽기용 편집본

원문: 1_기술이해.pdf · 294 쪽

원문 기준일: 2026-09-17 | 편집일: 2026-09-21

이 권의 흐름

8 시계열 데이터베이스 — 저장 모델·압축·카디널리티·InfluxDB

9 SCADA 와 관제 — FUXA·Grafana·운영 감시·구현 경계

전체 연결 — 6 개 고장·재측정·전체 측정 기록

원리 → 실제 구현 → 예외·한계 → 대안의 순서를 유지합니다.

같은 표·설정·설명은 정본 위치에 모으고, 원문의 설명 차이는 편집 주석으로 구분했습니다.

읽기 전에

이 문서는 첨부된 교본의 내용을 읽기 편하게 재편집한 것입니다. 원문에 인용된 저장소나 시스템을 새로 실행·검증하지 않았습니다. 제품 버전, 설정값, 라이선스 언급, 성능 수치는 원문 작성 시점의 설명이며 현재 상태에 대한 별도 검증이 아닙니다.

본문의 “설계 보고서”는 교본이 인용한 15 쪽 자료를 뜻합니다. “원문” 또는 “원문 교본”은 첨부된 294 쪽 PDF 입니다. “원문의 별도 실습서”는 이 편집본의 2 권과 다른 자료입니다. 원문의 소절 번호를 가리키는 표현에는 “원문”을 붙였고, 장 번호 1~9는 그대로 유지했습니다.

원문에는 같은 주제가 다른 측정 구간이나 구현 단계에서 다시 등장합니다. 이런 수치·조건·예외는 단순 중복으로 지우지 않았습니다. 특히 설계 요구와 구현 완료, 상태의 정확히 한 번 처리와 외부 출력 중복, 데이터 감사 추적과 명령 감사 로그를 구분합니다.

이 권의 입력: raw 관측, clean 정제값, anomaly score, alerts 입니다. 이를 저장하는 방식과 사람이 보는 화면, 명령의 실제 경로, 운영 감시의 역할을 구분해 읽습니다.

표·코드·도해의 출처는 해당 장의 원문 범위입니다. 별도의 편집 주석은 원문 쪽수를 함께 표시합니다. 코드는 원리와 설정을 읽기 위한 발췌이며, 중간 생략이나 줄바꿈이 있는 블록을 그대로 실행하는 설치 매뉴얼은 아닙니다.

8 시계열 데이터베이스 — 시간이 기본 키

원문 교본 234~263 쪽의 설명을 통합

Kafka 가 다시 읽을 수 있는 이벤트 로그라면 시계열 데이터베이스는 기간·태그·품질을 조건으로 이력을 조회하는 저장소입니다. 이 장에서는 시간 단위 저장과 압축, 카디널리티, InfluxDB 의 실제 스키마와 보존 정책을 연결합니다.

왜 일반 데이터베이스로는 안 되는가

데이터베이스(Database, DB)는 데이터를 저장하고 꺼내 주는 프로그램입니다. 우리가 흔히 말하는 데이터베이스는 대개 관계형 데이터베이스(Relational Database, RDB)입니다. 표(테이블)로 데이터를 담고, 행(row)과 열(column)로 구성하며, SQL 이라는 질의 언어로 꺼냅니다. PostgreSQL, MySQL, Oracle 이 여기에 속합니다.

관계형 DB 는 은행 계좌 같은 데이터를 위해 설계됐습니다. 계좌는 개수가 정해져 있고(1 만 개면 1 만 행), 잔액은 계속 갱신되며, 조회는 보통 "김철수의 계좌 한 건"처럼 특정 행 하나를 꼭 집어 찾습니다. 그래서 관계형 DB 의 핵심 장치는 B-트리 색인(B-tree index)입니다. 책 뒤의 찾아보기처럼, 값을 정렬해 두고 원하는 한 건으로 바로 점프하게 해 주는 자료구조입니다.

센서 데이터는 정반대입니다. 세 가지가 다릅니다.

첫째, 쓰기가 끝없는 덧불이이다. 센서는 값을 고치지 않습니다. 새 시각의 새 값을 계속 뒤에 붙일 뿐 입니다. 이것을 추가 전용(append-only) 쓰기라고 부릅니다. 하루면 수백만 행, 한 달이면 수억 행이 됩니다. 행 수가 정해진 계좌 테이블과는 규모의 성격 자체가 다릅니다.

둘째, 읽기가 긴 범위 집게다. "2026 년 9 월 14 일 03 시 12 분 07 초의 TT-101 값"을 묻는 사람은 사고 조사 때 말고는 없습니다. 실제 질문은 "지난 7 일간 반응기 온도 시간별 평균", "이번 달 압력 최댓값", "펌프 전류가 정격을 넘긴 구간 전부"입니다. 수천만 행을 훑어 요약 숫자 몇 개를 내놓는 작업입니다.

셋째, 갱신과 삭제가 거의 없다. 대신 통째로 오래된 것 버리기가 있습니다. "90 일 지난 원본은 지운다" 같은 규칙입니다.

관계형 DB 에 센서를 부으면 이 세 가지가 전부 불리하게 작동합니다. 무엇이 실제로 막히는지 순서대로 봅니다.

- 색인이 메모리를 넘어선다. 데이터가 커지면 B-트리 색인 자체가 수 기가바이트가 되어 메모리에 다 올라가지 않습니다. 그러면 삽입 한 건마다 디스크의 아무 데나 찔러 보는 랜덤 I/O 가 생깁니다. 순서대로 죽 읽는 것보다 수십 배 느린 접근 방식입니다.

- 보조 색인이 무작위로 흩어진다. 시각 색인은 그래도 오른쪽 끝에만 붙이지만, "설비별" "태그별"로 걸어 둔 보조 색인은 들어오는 값마다 전혀 다른 위치에 꽂힙니다. 여기서 랜덤 I/O 가 제대로 터집니다.

- 오래된 것 삭제가 통째로 안 된다. "90 일 지난 것 삭제"가 DELETE 수억 건이 되어 몇 시간씩 걸립니다.

· (PostgreSQL의 경우) 지운 자리가 바로 돌아오지 않는다. PostgreSQL은 MVCC(Multi-Version Concurrency Control, 다중 버전 동시성 제어)라는 방식을 씁니다. 행을 지우거나 고쳐도 옛 버전을 그 자리에 남겨 두고 새 버전을 따로 쓰는 구조라, 아무도 안 보게 된 옛 버전을 나중에 VACUUM이라는 정리 작업이 회수합니다. 대량 삭제를 하면 이 정리 작업이 통째로 뒤따라 붙습니다. 이건 관계형 DB 전체의 성질이 아니라 PostgreSQL의 구현 특성입니다. MySQL(InnoDB)이나 Oracle은 옛 버전을 별도 의 되돌리기 영역에 두기 때문에 동작이 다릅니다.

주의. 흔히 "행마다 B-트리를 다시 균형 맞추느라 느리다"고 설명하지만 이건 원인 지목이 틀렸습니다. 센서 데이터처럼 시각이 계속 커지지만 하는 추가 전용 쓰기는 B-트리의 오른쪽 맨 끝에만 붙습니다. 트리를 흔들어 재배치할 일이 거의 없고, PostgreSQL은 이 "맨 끝에 붙이기" 경우를 아예 따로 빠른 길로 처리합니다. 진짜 병목은 위에 적은 네 가지 — 색인이 메모리를 넘어서며 생기는 랜덤 I/O, 보조 색인의 무작위 삽입, 대량 DELETE, 그리고 PostgreSQL이라면 MVCC 팽창과 VACUUM입니다.

시계열 데이터베이스(Time-Series Database, TSDB)는 이 세 가지 패턴만을 위해 다시 설계한 데이터베이스입니다. 산업 현장에서는 같은 물건을 프로세스 히스토리안(Process Historian)이라고도 부릅니다. "공정 이력 보관소"라는 뜻이고, 이 교본의 설계 보고서도 이 용어를 씁니다.

패턴 비교 — 같은 "데이터베이스"인데 요구가 정반대다

항목	관계형 DB가 잘하는 일 (예: 주문 테이블)	시계열 DB가 받아야 하는 일 (예: 센서)
쓰기 패턴	여기저기 행을 삽입·수정. 순서 없음	시각 순서대로 뒤에 덧붙이기만. 사실상 추가 전용
쓰기 양	초당 수십~수천 건	초당 수만~수십만 건이 끊이지 않고 계속
읽기 패턴	특정 한 건 또는 소수 행 조회	넓은 시간 구간을 훑어 평균·최대·개수로 요약
갱신	빈번 (잔액 변경, 주소 수정)	거의 없음. 있으면 사고 조사용 예외
삭제	행 단위로 드물게	"N일 지난 것 전부" — 시간 구간 단위로 통째
중요한 축	기본 키(주문번호, 계좌번호)	시각. 시간이 사실상 기본 키다

시계열 DB가 다르게 하는 여섯 가지

1) 시간 기반 청크 분할 (time-based chunking / partitioning) 하나의 테이블을 시간 구간별로 자동으로 잘라 여러 개의 내부 조각으로 나눕니다. 이 조각을 청크 (chunk) 또는 파티션(partition)이라고 부릅니다. 예를 들어 하루 단위로 자르면 9월 14일 데이터는 9월 14일 청크에만 들어갑니다.

여기서 말하는 파티션은 "DB 테이블을 시간으로 자른 조각"입니다. 뒤에 Kafka 이야기가 나오면 똑같은 단어가 "토픽을 여러 줄로 쪼갠 조각"이라는 전혀 다른 뜻으로 쓰입니다. 두 파티션은 아무 관계가 없습니다. 이 장에서는 헷갈릴 만한 자리마다 "DB 청크" / "Kafka 파티션"이라고 붙여 적겠습니다.

청크 분할이 세 가지를 동시에 해결합니다. 첫째, 새 데이터는 항상 가장 최근 청크 하나에만 들어가므로 그 청크의 색인만 메모리에 두면 됩니다. 나머지 수백 개 청크는 디스크에서 잠들어 있어도 됩니다. 앞에서 본 "색인이 메모리를 넘는다"는 문제가 여기서 풀립니다. 둘째, "지난 3일 평균"을 물으면 해당 청크 세 개만 열면 되고 나머지는 아예 건드리지 않습니다. 이것을 청크 배제(chunk exclusion)라고 합니다. 셋째, 보존 기간이 지난 데이터는 DELETE 수억 건이 아니라 청크 파일 하나를 지우는 것으로 끝납니다. 몇 시간 걸리던 일이 1초가 됩니다.

2) 태그 색인 (tag index) 시계열 DB는 한 줄의 데이터를 네 부분으로 나눠 봅니다.

- measurement — 어느 표에 담긴 값인가. 관계형 DB의 테이블 이름에 해당합니다.
- 시각(timestamp) — 언제 찍혔나
- 태그(tag) — 이 값이 누구의 것인가. 설비명, 센서 태그명, 공장 이름, 라인 번호. 문자열이고 종류가 한정돼 있습니다.
- 필드(field) — 실제 측정값. 72.43 같은 숫자.

여기서 결정적인 설계가 나옵니다. measurement와 태그만 색인하고 필드는 색인하지 않습니다. 왜냐면 "TT-101의 데이터를 달라"는 질문은 자주 나오지만 "값이 정확히 72.43인 행을 달라"는 질문은 나오지 않기 때문입니다. 필드를 색인에서 빼면 색인 크기가 수십~수백 배 작아집니다.

measurement + 태그 조합이 같은 데이터를 하나로 묶은 것을 시리즈(series)라고 부릅니다. 여기서 measurement를 빼먹으면 안 됩니다. 시리즈를 결정하는 키는 measurement 이름으로 시작하고, 그 뒤에 태그들이 붙습니다. 그래서

태그 조합이 완전히 똑같아도 measurement 가 다르면 서로 다른 시리즈입니다. 뒤에서 보겠지만 AR-100 의 process 와 process_raw 가 정확히 이 경우입니다. 시계열 DB 의 내부 구조는 결국 시리즈 목록 + 시리즈별 값 배열입니다.

3) 압축 (compression) 시리즈로 묶어 같은 종류의 값을 나란히 놓으면 압축이 극적으로 잘 됩니다. 두 가지 기법이 표준이고, 둘 다 페이스북의 Gorilla 논문에서 정리된 방식입니다.

시각은 델타-오브-델타(delta-of-delta)로 줄입니다. 값 자체 대신 직전 값과의 차이를 적고(델타), 다시 그 차이끼리의 차이를 적습니다. 1 초 간격으로 규칙적으로 찍히는 센서라면 시각의 차이가 항상 1 초 이고, 그 차이의 차이는 0 입니다. 0 은 1 비트면 표현됩니다. 8 바이트(64 비트)짜리 타임스탬프가 사실상 몇 비트로 줄어듭니다. 다만 전제가 있습니다. 이 기법이 극적으로 먹히는 것은 샘플 간격이 규칙적일 때입니다. 설계 보고서는 산업 현장의 센서 데이터를 "공정 상황에 따라 밀리초 단위로 자율 푸시(Push)되는 불규칙한 이벤트 스트림"이라고 서술합니다. 정말로 불규칙하게 밀려 들어오면 차이의 차이가 0 이 아니라 매번 다른 값이되어 이 압축의 이득이 크게 줄어듭니다. AR-100 처럼 스캔 주기를 1 초로 고정한 구성에서는 규칙적이지만, 실제 현장 데이터에 그대로 대입하면 안 됩니다.

값은 직전 값과 XOR 을 취해 줄입니다. XOR(exclusive or, 배타적 논리합)은 두 비트를 자리마다 비교 해서 다르면 1, 같으면 0 을 내놓는 연산입니다. 그러니 두 수를 XOR 하면 "어느 자리가 달라졌는가"만 남습니다. 완전히 같은 두 수를 XOR 하면 결과는 0 이 됩니다. 소수점 숫자는 double(배정밀도 부동소수점) 형식으로 64 비트에 담깁니다. 부호 1 비트 + 지수 11 비트 + 가수(유효숫자) 52 비트 구조입니다. 값이 조금 변하면 앞쪽 부호·지수와 가수의 앞부분은 그대로 고 가수의 뒷부분이 바뀝니다. 여기서 흔히 "값이 조금 변하면 XOR 이 거의 0 이 된다"고 설명하는데, 그건 사실이 아닙니다. 실제로 계산해 보면 이렇습니다.

```
72.43 → 0x40521B851EB851EC
72.44 → 0x40521C28F5C28F5C
XOR   → 0x000007ADEB7ADEB0
선행 0 : 21 비트
1의 개수 : 64 비트 중 28 개 ← "거의 0" 이 아니다
유효 비트 창 : 39 비트 (맨 앞 1 부터 맨 뒤 1 까지)
```

64 비트 중 28 개가 1 입니다. 그럼 왜 압축이 되느냐 — 이득이 나오는 자리는 "0 이 많아서"가 아니라 앞뒤로 잘라 낼 수 있어서입니다. Gorilla 는 XOR 결과를 통째로 적는 대신 선행 0 이 몇 개인지(5 비트) 와 유효 비트 창의 길이(6 비트)만 기록하고 그 창 안의 비트만 저장합니다. 위 예라면 제어비트 2 + 5 + 6 + 39 = 약 52 비트. 64 비트에서 52 비트로, 적당한 절약입니다. 최대 효과는 값이 완전히 같을 때 납니다. XOR 이 0 이면 "직전과 같음"을 뜻하는 1 비트로 끝납니다. 64 비트가 1 비트가 됩니다. 안정 운전 중인 공정 신호는 같은 값이나 거의 같은 값이 길게 이어지므로, 실무에서 10 배 이상의 압축이 흔히 나오는 이유가 여기 있습니다.

주의. "관계형 DB 는 이런 압축이 불가능하다"는 말은 틀렸습니다. 한계는 관계형 모델에 있는 게 아니라 행 저장(row store) 방식에 있습니다. 한 행 안에는 시각·문자열·숫자가 섞여 있어서 같은 종류의 값을 이어 붙일 수가 없습니다. 그래서 뒤에 나올 TimescaleDB 는 오래된 청크를 열 단위로 다시 묶어(컬럼 지향) 압축합니다. 같은 PostgreSQL 위에서도 10~20 배가 납니다 (Timescale 이 문서로 밝히는 범위. 이 프로젝트에서 켜 값이 아닙니다). 압축을 못 하게 만드는 것은 SQL 이나 테이블 모델이 아니라 저장 배치 방식입니다.

4) 다운샘플링 (downsampling) / 롤업 (rollup) 1 초 간격 원본을 영원히 들고 있을 이유는 없습니다. 3 년 전 데이터를 1 초 단위로 볼 사람은 없고, 있어도 화면에 그리지 못합니다. 그래서 오래된 데이터는 1 분 요약, 더 오래된 것은 1 시간 요약으로 줄여 저장하고 원본은 버립니다. 이것이 다운샘플링이고, 산업계 용어로는 롤업입니다.

얼마나 줄어드는지는 요약값을 몇 개 남기느냐에 달렸습니다. 1 초 간격 원본 60 점이 1 분 요약 1 행이 되지만, 그 1 행에 무엇을 담느냐로 달라집니다.

· 평균 하나만 남기면 값 60 개 → 1 개, 60 분의 1

· 평균과 최댓값을 남기면 값 60 개 → 2 개, 30 분의 1

· 실무에서 보통 쓰는 min·max·avg·count 네 개를 남기면 15 분의 1 그리고 요약 행에도 시각과 태그가 붙으므로 실제 저장량 감소는 이 비율보다 조금 작습니다. "다운샘플링하면 60 분의 1"이라고 외우면 틀립니다. 공정 데이터에서는 최댓값·최솟값이 사고 조사의 핵심이라 평균만 남기는 경우가 오히려 드뭅니다.

중요한 구분: 다운샘플링은 저장할 때 줄이는 것과 조회할 때 줄이는 것 두 가지가 있습니다. 조회 시점 다운샘플링은 원본을 그대로 두고 화면에 그릴 때만 요약하는 것이고, 저장 시점 다운샘플링은 아예 요약본만 남기고 원본을 버리는 것입니다. 둘 다 씁니다.

5) 보존 정책 (retention policy) "이 데이터는 며칠 지나면 자동으로 지운다"는 규칙을 DB에 등록해 두는 것입니다. 사람이 청소 스크립트를 짜서 돌리는 게 아니라 엔진이 알아서 합니다. 앞서 본 청크 분할 덕분에 이 삭제가 청크 하나 지우기로 끝납니다. 보통 계단식으로 씁니다 — 원본 30 일 / 1 분 요약 2 년 / 1 시간 요약 10 년.

6) 조회 시점에 결측 구간을 메워 주는 함수 (gap filling) 이게 이 장에서 가장 중요한 항목인데 잘 알려져 있지 않습니다. 센서 데이터에는 반드시 구멍이 생깁니다. 통신이 끊기거나, 센서가 잠깐 죽거나, 패킷이 유실됩니다. 그런데 시계열 DB는 그 구멍을 쿼리 를 실행하는 순간에 메워 주는 함수를 스스로 갖고 있습니다. 저장된 데이터는 건드리지 않고, 결과만 채워서 돌려줍니다.

채우는 방법이 두 가지입니다.

- LOCF — Last Observation Carried Forward, "직전 유효값 유지"입니다. 마지막으로 제대로 들어온 값을 구멍 끝까지 그대로 끌고 갑니다. 계산이 거의 없어 즉시 적용되지만, 값이 변하고 있던 추세가 통째로 끊겨 평평한 선이 됩니다.

- 선형 보간(linear interpolation) — 구멍 양쪽 끝의 두 값을 직선으로 이어 그 사이를 시간차에 비례 해 채웁니다. 온도·레벨처럼 연속적으로 변하는 물리량에는 물리적으로 타당하지만, 구멍 뒤쪽 값이 도착할 때까지 기다려야 계산이 됩니다.

엔진마다 함수 이름이 다릅니다. InfluxDB 2.x의 Flux에서는 fill(usePrevious: true) , TimescaleDB에서는 time_bucket_gapfill() 과 함께 locf() 또는 interpolate() 를 씁니다.

이것이 왜 중요한가. 설계 보고서가 여기서 아주 구체적인 판단을 내리기 때문입니다.

"머신러닝이나 딥러닝 기반 분석을 수행하지 않고 단순 모니터링 및 시각화 대시보드 운영만을 목표로 하는 경우, 데이터 파이프라인 내부에서 결측치를 인위적으로 보간할 당위성은 낮다. InfluxDB 나 TimescaleDB 같은 시계열 데이터베이스 엔진은 쿼리 실행 시점에 직전 값 유지(LOCF)나 선형 보간 함수를 자체적으로 수행하여 공백을 메우는 기능을 지원한다."

즉 화면에 그리기만 할 거라면 파이프라인 중간에서 값을 만들어 넣을 이유가 없습니다. 저장은 구멍 난 채로 하고, 그럴 때만 DB에게 메워 달라고 하면 됩니다. 설계 보고서가 곧바로 이어 붙이는 경고가 그 이유 입니다 — "원천 데이터 레벨에서 인위적으로 계산된 가공치를 덮어쓰게 되면 실제 물리 계측치와 추정치 간의 경계가 모호해져 향후 사고 발생 시 감사 추적(Audit Trail)의 법적·기술적 신뢰성을 훼손할 수 있다."

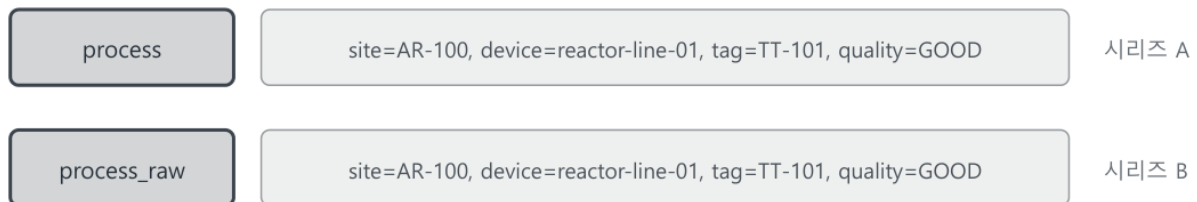
반대로 머신러닝을 붙이면 이야기가 달라집니다. 신경망은 고정된 크기의 완전한 숫자 행렬을 요구하므로 구멍이 있으면 아예 계산이 안 됩니다. 그래서 설계 보고서는 ML을 붙이는 경우에만 파이프라인 안에서 의 보간을 "필수 전처리 공정"이라고 못 박습니다. AR-100 이 파이프라인 안에서 보간을 하는 이유가 정확히 이것이고, 그러면서도 감사 추적을 지키려고 어떤 구조를 택했는지는 뒤에서 봅니다.

하이퍼테이블 (밖에서는 테이블 하나로 보인다)



시간 기반 청크 분할. 쓰기·조회·삭제 세 가지 문제가 같은 장치 하나로 풀린다. 여기서 말하는 조각은 DB 청크이지 Kafka 파티션이 아니다.

시리즈 키 = **measurement** + 태그 집합



태그 집합은 글자 하나까지 똑같다
그래도 measurement 가 다르므로 서로 다른 시리즈다

원문 도해 · 시간 청크와 시리즈 키 (원문 240 쪽)

카디널리티, 그리고 그것이 폭발하면 죽는 이유

카디널리티(cardinality)는 원래 수학에서 "집합의 원소 개수"를 뜻하는 말입니다. 시계열 DB에서는 뜻이 아주 구체적입니다.

카디널리티 = 이 데이터베이스가 들고 있는 서로 다른 시리즈의 개수

시리즈는 measurement + 태그 조합이라고 했습니다. 그러니 세는 방법이 정해집니다.

① **measurement** 하나를 고른다 → ② 그 **measurement**의 태그들이 갖는 값의 가짓수를 전부 곱한다 → ③ **measurement** 별로 나온 수를 더한다

measurement 안에서는 곱하기, measurement 사이에서는 더하기입니다. 문제를 일으키는 쪽은 언제나 곱하기입니다.

예를 들어 공장 1 곳 × 설비 1 대 × 센서 태그 12 개 × 품질 표기 3 종 = 36. 이 measurement의 시리즈가 36개입니다. 아주 작습니다. 그런데 여기에 "센서 일련번호"라는 태그를 하나 더 붙이고 그 값이 5만 종이라면 36 × 50,000 = 180 만이 됩니다. 태그 하나 추가했을 뿐인데 시리즈 수가 5만 배가 됐습니다. 이것이 카디널리티 폭발(cardinality explosion)입니다.

왜 폭발하면 시스템이 죽는가. 시리즈 하나마다 DB가 들고 있어야 하는 것이 있습니다.

· 색인 항목 — "이 measurement의 이 태그 조합은 몇 번 시리즈다"라는 대응표. 조회 속도를 위해 상당 부분이 메모리에 상주합니다.

· 쓰기 버퍼 — 시리즈마다 값을 모았다가 디스크에 내려쓰는 작은 메모리 공간.

· 파일 구조 — 시리즈 단위로 압축 블록이 만들어지므로, 시리즈가 많으면 블록이 잘게 쪼개져 압축률이 떨어지고 파일 핸들도 늘어납니다.

시리즈가 180 만 개면 색인만으로 수 기가바이트를 먹습니다. 메모리를 넘어서는 순간 운영체제가 프로세스를 강제 종료합니다. 이것을 OOM(Out Of Memory, 메모리 고갈)이라고 합니다. 데이터베이스 가 느려지는 게 아니라 그냥 죽습니다. 그리고 다시 켜면 색인을 다시 만들면서 또 죽습니다.

초보가 반드시 헷갈리는 지점. "데이터가 많아서 죽는다"와 "카디널리티가 높아서 죽는다"는 완전히 다른 문제입니다. 시리즈 36 개에 100 억 건이 들어와도 시계열 DB 는 잘 견딥니다 — 압축이 잘 되고 색인은 36 줄이니까요. 반대로 시리즈 500 만 개에 각각 3 건씩만 들어와도 죽습니다. 총 1,500 만 건, 데이터로 보면 작습니다. 죽이는 것은 데이터의 양이 아니라 태그 조합의 가짓수입니다.

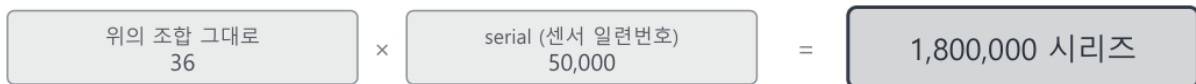
그리고 이 말은 시계열 DB 전체에 해당합니다. 뒤에서 Prometheus 가 카디널리티에 특히 약하다는 이야기를 하겠지만, InfluxDB 나 TimescaleDB 를 쓴다고 면역이 되는 게 아닙니다. 버티는 범위가 다를 뿐입니다.

한 measurement 안의 카디널리티 = 태그 값 가짓수의 곱

설계가 건강할 때 — measurement process



태그를 하나 잘못 붙였을 때



데이터 건수는 그대로인데 색인이 5만 배가 된다 — 이것이 카디널리티 폭발

원문 도해 · 태그 조합과 카디널리티 증가 (원문 242 쪽)

주의. 태그에 절대 넣으면 안 되는 값에는 규칙이 있습니다 — 매년 달라지는 값입니다. 타임스탬프, 요청 ID, UUID, 사용자 ID, 측정값 자체, 에러 메시지 전문, 백분율이 섞인 설명 문자열. 이런 것을 태그로 붙이면 데이터가 한 줄 들어올 때마다 새 시리즈가 하나 생깁니다. 즉 카디널리티가 데이터 건수와 같아집니다. 이런 값은 태그가 아니라 필드로 넣어야 합니다. 필드는 색인되지 않으므로 아무리 다양해도 시리즈 수를 늘리지 않습니다. 태그는 "그룹으로 묶어서 조회할 대상"만, 필드는 "그냥 기록만 할 값"입니다.

TimescaleDB: PostgreSQL 을 시계열 DB 로 만드는 확장

TimescaleDB 는 독립된 데이터베이스가 아니라 PostgreSQL 의 확장(extension)입니다. 확장이란 PostgreSQL 에 추가로 설치해 기능을 얻는 플러그인을 말합니다. CREATE EXTENSION timescaledb; 한 줄로 켵니다. 그러니까 TimescaleDB 를 쓴다는 것은 여전히 PostgreSQL 을 쓰는 것입니다. 장치가 세 개입니다.

장치 ① 하이퍼테이블(hypertable). 겉보기에는 평범한 PostgreSQL 테이블 하나인데, 내부적으로는 앞서 설명한 시간 기반 청크로 자동 분할되어 있습니다. 데이터를 넣으면 시각을 보고 알맞은 청크에 자동으로 들어가고, 조회하면 필요한 청크만 열립니다. 사용자는 그 사실을 알 필요가 없습니다.

```
-- 평범한 테이블을 만들고
CREATE TABLE process (
  ts      TIMESTAMPTZ NOT NULL,      -- 시간대 정보를 함께 갖는 시각 (마이크로초까지)
  site    TEXT,
  device  TEXT,
  tag     TEXT,
  quality TEXT,
  value   DOUBLE PRECISION          -- 64 비트 부동소수점 (배정밀도 실수)
);
-- 하이퍼테이블로 바꾼다. chunk_time_interval = 청크 하나가 담을 시간 폭
SELECT create_hypertable('process', 'ts', chunk_time_interval => INTERVAL '7 days');
-- 90 일 지난 청크는 자동 삭제
SELECT add_retention_policy('process', INTERVAL '90 days');
```

TIMESTAMPTZ 는 timestamp with time zone 의 줄임말로, 시간대 정보를 함께 갖는 시각 타입입니다. 정밀도는 마이크로초(100 만분의 1 초)까지입니다. DOUBLE PRECISION 은 앞에서 본 64 비트 double 이고, 대부분의 프로그래밍 언어에서 말하는 double 과 같습니다.

장치 ② 연속 집계(continuous aggregate). 이름이 어렵지만 뜻은 간단합니다. "1 분 평균"을 미리 계산해 따로 저장해 두는 표인데, 새 데이터가 들어오면 바뀐 구간만 다시 계산해서 갱신합니다. 전체를 다시 계산하지 않습니다. 일반 데이터베이스의 구체화 뷰(materialized view)는 갱신할 때 전체를 다시 계산해야 하는데, 연속 집계는 증분으로 갱신하기 때문에 끝없이 자라는 시계열에 쓸 수 있습니다. 이것이 저장 시점 다운샘플링의 구현체입니다.

```
CREATE MATERIALIZED VIEW process_1m
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 minute', ts) AS bucket,    -- 시각을 1분 단위로 내려 묶음 라벨을 만든다
       tag,
       avg(value) AS avg_value,
       max(value) AS max_value
FROM process
GROUP BY bucket, tag;
```

time_bucket('1 minute', ts) 는 시각을 1 분 단위로 내림해 같은 분에 속한 행들에게 같은 라벨을 붙이는 함수입니다. SQL 의 GROUP BY 가 그 라벨로 묶습니다. 위 예는 평균과 최대값 두 개를 남기므로 값의 개수 기준으로 30 분의 1 이 됩니다.

장치 ③ 컬럼 단위 압축(columnar compression). 앞에서 "행 저장 방식은 압축이 어렵다"고 했는데, TimescaleDB 는 오래된 청크를 열 단위로 다시 묶어 이 한계를 넘습니다. 같은 열의 값끼리 모아 놓으면 시각은 델타-오브-델타로, 반복되는 문자열은 사전 방식으로 줄일 수 있습니다.

```
ALTER TABLE process SET (
  timescaledb.compress,
  timescaledb.compress_segmentby = 'tag',    -- 태그별로 묶어서
  timescaledb.compress_orderby   = 'ts DESC' -- 시각 순으로 정렬해 저장
);
SELECT add_compression_policy('process', INTERVAL '7 days'); -- 7 일 지난 청크는 자동 압축
```

Timescale 이 문서로 밝히는 압축률은 10~20 배입니다(이 프로젝트에서 쟁 값이 아니라 제품 문서의 수치입니다). 여기서 확인해 둘 것은 같은 PostgreSQL 위에서 이 정도가 난다는 사실입니다. 압축을 가로막는 것은 관계형 모델이 아니라 저장 배치 방식이라는 앞의 설명이 여기서 확인됩니다.

TimescaleDB 의 진짜 강점은 "SQL 그대로"입니다. 조인(JOIN)이 됩니다. 설비 마스터 테이블, 작업지시 테이블, 품질검사 테이블과 센서 데이터를 한 쿼리에서 묶을 수 있습니다.

마스터 데이터(master data)란 자주 변하지 않는 기준 정보를 말합니다. 설비 목록과 사양, 제품 코드, 규격 상·하한, 작업지시 번호, 검사 항목 같은 것입니다. "지금 이 순간의 측정값"인 시계열 데이터와 대비되는 개념입니다. 실무의 질문은 대개 둘을 같이 봐야 답이 나옵니다 — "지난달 A 제품을 만든 작업지시 구간에서만 반응기 온도 평균을 내라" 같은 질문에는 작업지시 마스터와 센서 시계열이 동시에 필요합니다.

그 밖에 기존 PostgreSQL 드라이버, 기존 BI 도구, 기존 백업 절차, 기존 권한 체계가 전부 그대로 통합니다. 트랜잭션도 됩니다. 회사에 PostgreSQL 을 다룰 줄 아는 사람이 이미 있다면 학습 비용이 사실상 0 입니다.

대가도 있습니다. PostgreSQL 위에 얹는 구조이므로 PostgreSQL 의 운영 특성(앞서 본 MVCC 와 VACUUM , 커넥션 관리, 연결 수 제한)을 그대로 물려받습니다. 운영해 본 적 없는 팀에게는 그 자체가 새 학습 비용입니다. 반면 "전용 엔진보다 쓰기 처리량이 떨어진다"는 식의 단점은 이 교본의 근거로는 할 수 없습니다. 오히려 PDF 본문은 "InfluxDB(또는 대규모 분석 워크로드에 최적화된 TimescaleDB)"라고 써서 TimescaleDB 를 대규모 쪽에 놓습니다. 둘 중 무엇이 빠른지는 스키마·하드웨어·쿼리 형태에 따라 갈리며, 이 프로젝트는 TimescaleDB 를 구동해 보지 않았으므로 비교 실측이 없습니다.

InfluxDB: 시계열만을 위해 처음부터 만든 엔진

InfluxDB 는 시계열만을 위해 바닥부터 만든 전용 데이터베이스입니다. 1.x 와 2.x 의 저장 엔진 이름이 TSM(Time-Structured Merge Tree)인데, 이름 그대로 "시간 구조로 정리된 병합 트리"입니다. 들어오는 데이터를 메모리에 모았다가 시리즈별로 정렬·압축해 디스크 파일로 내려쓰고, 그 파일들을 뒤에서 조용히 합칩니다. 관계형 DB 의 B-트리와는 완전히 다른 구조입니다. 시리즈 색인은 TSI(Time Series Index)라는 디스크 기반 색인으로 관리해, 색인을 전부 메모리에 올리는 방식보다 더 많은 시리즈를 버팁니다.

버전을 잘 봐야 합니다. TSM 은 1.x/2.x 의 엔진입니다. 3.x 에서는 저장 엔진이 Apache Arrow / Parquet/ DataFusion 기반으로 통째로 바뀌었으므로 "InfluxDB 는 TSM 엔진이다"라고 뭉뚱그리면 틀립니다. AR-100 이 쓰는 것은 2.7 이므로 이 장의 설명은 TSM 기준입니다.

InfluxDB의 데이터 모델은 네 조각입니다.

- measurement — 관계형 DB의 테이블 이름에 해당합니다. process, alerts 같은 것.
- 태그 집합(tag set) — 색인되는 문자열들. measurement와 함께 시리지를 결정합니다.
- 필드 집합(field set) — 색인 안 되는 실제 값들. 숫자·문자열·불리언 모두 됩니다.
- 타임스탬프 — 나노초 단위까지 저장 가능.

이것을 한 줄의 텍스트로 표현하는 문법이 라인 프로토콜(Line Protocol)입니다. InfluxDB에 데이터를 넣는 표준 방식이고, 형식은 이렇습니다.

측정이름, 태그키=태그값, 태그키=태그값 필드키=필드값, 필드키=필드값 타임스탬프

측정이름	태그 집합 (색인됨)	필드 집합 (색인 안 됨)	나노초
measurement	태그 집합 (색인됨)	필드 집합 (색인 안 됨)	나노초

구분 규칙은 세 가지입니다. measurement와 태그 집합 사이는 쉼표, 태그 집합과 필드 집합 사이는 공백, 필드 집합과 타임스탬프 사이도 공백입니다.

초보가 가장 많이 틀리는 지점이 여기입니다. 구분자로 쓰이는 문자가 데이터 안에 들어가면 앞에 역슬래시(\)를 붙여 탈출시켜야 하는데, 위치마다 탈출 대상이 다릅니다. "공백만 조심하면 된다"고 외우면 반드시 깨집니다.

위치	반드시 이스케이프해야 하는 문자
measurement 이름	쉼표, 공백
태그 키 · 태그 값 · 필드 키	쉼표, 등호 =, 공백
문자열 필드 값 (큰따옴표로 감쌈)	큰따옴표 " · 역슬래시 \

탈출 문자는 전부 역슬래시입니다. 그리고 줄바꿈은 줄 자체의 구분자이므로 값 안에 넣을 수 없습니다. 실제로 사고가 나는 자리는 대개 사람이 붙인 설명 문자열입니다. 알람 사유나 장비 모델명처럼 쉼표·등호·공백이 자연스럽게 들어가는 값을 태그로 넣었다가 줄이 깨집니다. 이런 값은 원래 태그가 아니라 필드에 넣어야 할 것이기도 합니다.

라인 프로토콜은 그냥 텍스트이므로 HTTP로 여러 줄을 한꺼번에 밀어 넣습니다. 한 건씩이 아니라 수천 건을 한 요청에 묶어 보내는 것을 벌크 쓰기(bulk write)라고 하고, 보내는 쪽이 자기 속도로 밀어 넣는 방식을 푸시(Push)라고 합니다. 뒤에 나올 Prometheus의 풀(Pull)과 대비되는 지점입니다.

나노초(nanosecond) 정밀도도 InfluxDB의 특징으로 자주 꼽힙니다. 1 나노초는 10억분의 1초입니다. 다만 이 숫자를 과대평가하지 않는 것이 중요합니다. 흔히 드는 이유 세 가지 — 같은 밀리초에 여러 값이 겹쳐 순서가 무너지는 것, 진동 센서처럼 수 킬로헤르츠로 샘플링하는 계측, 장비가 만든 원래 타임스탬프의 보존 — 은 전부 마이크로초면 충족됩니다. 수 kHz 샘플링도 간격이 수백 마이크로초 수준이고, TimescaleDB의 TIMESTAMPTZ도 마이크로초까지 갑니다. 나노초가 실제로 필요한 경우는 전력 계통 과도 현상 기록처럼 아주 특수한 계측에 한정됩니다. 나노초를 쓰는 대부분의 시스템은 "필요해 서"가 아니라 "포맷이 그렇게 정의돼 있어서" 씁니다. AR-100이 정확히 그 경우인데, 이유는 뒤 AR-100 절에서 봅니다.

조직(organization, org)은 InfluxDB 2.x에서 도입된 소유 단위입니다. 버킷·대시보드·태스크·토큰이 전부 어느 org에 속하는지로 갈라지고, 사용자 권한도 org 단위로 부여합니다. 회사 하나에 여러 팀이 같은 InfluxDB를 쓸 때 서로의 데이터를 못 보게 나누는 칸막이라고 생각하면 됩니다. 조회 API를 호출 할 때도 org 이름을 함께 넘겨야 합니다.

InfluxDB의 질의 언어는 버전에 따라 다릅니다. 1.x는 SQL 비슷한 InfluxQL, 2.x는 Flux라는 파이프라인형 언어, 3.x에서는 다시 SQL을 지원합니다. Flux는 > 기호로 단계를 이어 붙이는 방식이라 SQL과 문법이 많이 다르고, 이 점이 도입 시 가장 큰 진입 장벽입니다.

Flux 기본 함수 — 이 장의 쿼리를 읽기 위해 뒤에 나올 AR-100 대시보드 쿼리를 읽으려면 함수 여섯 개만 알면 됩니다. Flux는 왼쪽 결과를 >로 다음 함수에 넘기는 구조입니다. 공장의 컨베이어처럼 앞 단계의 출력이 뒷 단계의 입력이 됩니다.

함수	하는 일
from(bucket: "process")	어느 버킷에서 읽을지 정한다. 항상 맨 앞
range(start:, stop:)	시간 범위를 자른다. 생략할 수 없다 — 범위 없이 전체를 훑는 사고를 막는 안전장치

함수	하는 일
<code>filter(fn: (r) => ...)</code>	조건에 맞는 행만 남긴다. <code>r</code> 은 행 하나, <code>r._measurement · r._field · r["tag"]</code> 로 열을 읽는다
<code>group(columns: ["quality"])</code>	지정한 열의 값이 같은 행끼리 다시 묶는다. 이후 집계기가 이 묶음 단위로 돈다
<code>aggregateWindow(every: fn:, createEmpty:)</code>	시간을 <code>every</code> 폭의 창으로 자르고 각 창에 <code>fn</code> (평균 <code>mean</code> · 개수 <code>count</code> 등)을 적용한다
<code>keep(columns: [...])</code>	결과에서 지정한 열만 남긴다. 화면으로 보내는 데이터를 줄이는 용도

`aggregateWindow` 의 `createEmpty` 가 초보를 가장 많이 헷갈리게 합니다. 데이터가 한 건도 없는 창을 결과 에 남길 것인가를 정하는 옵션입니다. `false` 면 그 창을 아예 빼고, `true` 면 빈 값(집계 함수에 따라 0)으로 한 줄을 만들어 남깁니다. 어느 쪽이 맞는지는 그 패널이 무엇을 보여 주려는가에 달렸습니다 — AR-100 의 두 패널이 정반대로 설정된 이유를 뒤에서 봅니다.

TimescaleDB 와 InfluxDB — 어느 쪽을 고르나

기준	TimescaleDB	InfluxDB
정체	PostgreSQL 확장	시계열 전용 엔진
질의 언어	SQL 전부. 조인·서브쿼리·윈도우 함수 그대로	Flux (2.x) / InfluxQL (1.x) / SQL (3.x)
분할 장치	하이퍼테이블 (자동 청크)	샤드 그룹 (보존 정책 단위로 분할)
압축	오래된 청크를 열 단위로 재배치해 압축 (<code>add_compression_policy</code>)	시리즈별 블록에 델타-오브-델타 + XOR 압축
다운샘플링	연속 집계 (중분 갱신 구체화 뷰)	태스크 / 다운샘플링 잡
결측 구간 메우기	<code>time_bucket_gapfill</code> + <code>locf()</code> / <code>interpolate()</code>	Flux <code>fill(usePrevious: true)</code> 등
쓰기 방식	INSERT , COPY — 기존 드라이버 그대로	Line Protocol HTTP 푸시
타임스탬프	마이크로초 (PostgreSQL <code>TIMESTAMPTZ</code>)	나노초
관계형 데이터와 조인	같은 DB 안에서 자연스럽게	어려움. 애플리케이션에서 합쳐야
유리한 상황	이미 PostgreSQL 을 쓰고, 마스터 데이터와 함께 봐야 할 때	순수 텔레메트리(원격 계측 데이터)를 푸시로 부어 넣고 관계형 결합이 필요 없을 때

Prometheus 는 왜 이 자리에 오면 안 되는가

Prometheus 도 시간에 따른 숫자를 저장하는 데이터베이스입니다. 그래서 처음 배우는 사람은 "이것도 시계열 DB 니까 센서 데이터를 넣으면 되겠네"라고 생각합니다. 설계 보고서는 이 오해를 "파이프라인 설계에서 가장 심각한 병목과 장애를 초래하는 원인 중 하나"라고 못 박습니다.

Prometheus 는 IT 인프라 감시를 위해 만들어졌습니다. 서버가 살아 있나, CPU 가 몇 퍼센트인가, 요청 이 초당 몇 건인가. 이 목적이 설계 전부를 결정했습니다.

수집 방식이 풀(Pull)입니다. Prometheus 가 감시 대상의 `/metrics` 주소를 정해진 주기로 직접 찾아가서 긁어 옵니다. 이것을 스크랩(scrape)이라고 합니다. 인프라 감시에서는 이게 장점입니다 — 감시 대상이 죽으면 스크랩이 실패하니 죽은 것을 바로 압니다. 하지만 센서에는 치명적입니다. 스크랩 주기가 10 초라면 그 사이 10 초 동안 센서가 찍은 값들은 영원히 사라집니다. 굵은 순간의 값 하나만 남습니다. 공정 데이터는 "굵으려 갔을 때의 값"이 아니라 "매 순간 찍힌 값 전부"여야 합니다. PDF 표현 그대로 타임스탬프 해상도가 "스크랩 주기에 종속적"입니다.

게다가 설계 보고서는 산업 센서 데이터를 "공정 상황에 따라 밀리초 단위로 자율 푸시(Push)되는 불규칙한 이벤트 스트림"이라고 규정합니다. 스스로 밀려 나오는 데이터를 주기적으로 굵으려 가는 구조로 받으면 구조적으로 맞지 않습니다.

저장 타입이 Float64 하나뿐입니다. 64 비트 실수 하나만 저장합니다. 장비 에러 코드 문자열, 운전 모드 이름, 다차원 JSON 페이로드를 담을 수 없습니다. 산업 현장 데이터는 숫자만이 아닙니다.

그리고 카디널리티입니다. Prometheus는 시리즈 색인을 전부 메모리에 올려 두는 구조입니다. 그래서 시리즈 수가 늘면 인덱스가 RAM 용량을 넘어서고 프로세스가 OOM으로 충돌합니다. 센서 데이터는 정확히 그 반대 성질을 가집니다 — 센서 일련번호, 공정 라인 번호, 설비 태그가 조합되면 수백만 개 이상의 시리즈가 나옵니다. 설계 보고서의 표현대로 "극단적인 고 카디널리티 구조"입니다.

주의 — 여기를 반대로 읽으면 안 됩니다. PDF 비교표의 "시계열 전용 스토리지 엔진 기반 고 카디널리티 최적화"는 정도 차이를 말하는 것이지 면역을 말하는 것이 아닙니다. InfluxDB OSS도 고카디널리티에서 OOM으로 죽습니다. 앞의 카디널리티 설명에서 "시리즈 500만 개면 죽는다"고 한 것은 시계열 DB 일반에 대한 이야기였고, 이 장 마지막에서 AR-100 자신의 top_contributors 태그를 카디널리티 폭발의 전형으로 경고하는 것도 그 InfluxDB 위에서 벌어지는 일입니다. InfluxDB의 TSI는 색인을 디스크에 두어 Prometheus보다 버티는 범위가 넓지만, 그뿐입니다. "InfluxDB면 태그를 아무렇게나 붙여도 된다"는 결론은 이 표에서 나올 수 없습니다.

그래서 결론은 둘을 섞는 게 아니라 철저히 이원화하는 것입니다. 센서 데이터는 InfluxDB나 TimescaleDB로, 파이프라인 자체의 건강 상태는 Prometheus로. 둘은 경쟁 관계가 아니라 다른 일을 하는 다른 도구입니다.

Prometheus가 보는 숫자는 어떤 것인가. 설계 보고서가 예로 드는 것이 컨슈머 랙과 체크포인트 소요 시간인데, 둘 다 이름만 봐서는 무슨 값인지 알 수 없으니 풀어 둡니다.

· Kafka 컨슈머 랙(consumer lag) — 쌓인 만큼과 읽은 만큼의 차이입니다. Kafka는 토픽의 각 파티션에 들어온 메시지에 0, 1, 2... 하고 번호(오프셋)를 붙입니다. 랙 = "이 파티션에 마지막으로 쌓인 번호" - "이 컨슈머 그룹이 읽어 간 마지막 번호". 단위는 건수입니다. 0에 가까우면 실시간으로 따라잡고 있는 것이고, 계속 커지면 소비가 생산을 못 따라가고 있다는 뜻입니다. 메트릭 이름 kafka_consumergroup_lag가 바로 이 값이고, AR-100의 경보 규칙은 sum by (consumergroup, topic) (kafka_consumergroup_lag) > 5000 이 2분 이상 지속되면 울립니다.

· Flink 체크포인트(checkpoint) — Flink가 처리 도중의 상태 전체를 주기적으로 스냅샷으로 떼서 저장하는 것입니다. 이게 있어야 프로세스가 죽어도 마지막 스냅샷에서 정확히 이어서 재개할 수 있습니다. 메트릭 flink_jobmanager_job_lastCheckpointDuration은 가장 최근 스냅샷을 뜨는 데 걸린 시간(밀리초)입니다. AR-100의 체크포인트 간격은 10초로 설정돼 있는데, 소요 시간이 그 간격에 근접하면 다음 체크포인트가 밀리기 시작한다는 신호입니다.

설계 보고서의 Prometheus 대 InfluxDB 비교표 (원문 전문)

분석 지표	Prometheus	InfluxDB
시스템 본질적 목적	IT 인프라, 컨테이너, 서비스 계층 모니터링 및 실시간 알림	대규모 산업 센서 텔레메트리 적재, 고성능 프로세스 히스토리안
데이터 수집 아키텍처	Pull 메커니즘 (정해진 주기의 HTTP 타깃 스크랩)	Push 메커니즘 (Line Protocol 기반 비동기 벌크 쓰기)
지원 데이터 타입	단일 Float64 (문자열, 복합 구조체, 바이너리 저장 불가)	Float, Integer, String, Boolean, 다차원 필드셋
카디널리티 수용 한계	고 카디널리티에 극도로 취약 (메모리 고갈 및 인덱스 붕괴 위험)	시계열 전용 스토리지 엔진 기반 고 카디널리티 최적화
타임스탬프 해상도	밀리초(Millisecond) 단위, 스크랩 주기에 종속적	나노초(Nanosecond) 단위 정밀 타임스탬프 지원
데이터 보존 주기	단기~중기 (기본 수일~수개월, 인프라 메트릭용)	장기 영구 보관 (수년 이상의 연속 공정 이력 보존 및 다운샘플링)

설계 보고서는 이 표 아래에 결론을 못 박습니다. "Prometheus를 센서 데이터 수집의 영구 저장소로 채택하는 것은 불가능하며, 두 컴포넌트는 철저히 이원화되어 운영되어야 한다." 그리고 최종 권고에서 다시 한번 "시계열 센서 데이터 히스토리안은 InfluxDB에 위임하고 Prometheus는 순수 파이프라인 인프라 모니터링으로 격리해야 한다"고 씁니다. 다만 표의 네 번째 줄은 Prometheus 대비 상대적 우위로 읽어야 하며, 절대적 안전을 뜻하지 않는다는 점을 다시 적어 둡니다.

Kafka Connect와 Telegraf: 코드 없이 잇는 두 가지 방법

Kafka에 쌓인 데이터를 시계열 DB에 넣으려면 누군가 꺼내서 옮기는 일을 해야 합니다. 직접 짜면 이런 코드가 됩니다 — Kafka 컨슈머를 만들고, 메시지를 받아 JSON을 파싱하고, 필드를 골라내고, 라인 프로토콜로 바꾸고, InfluxDB에

HTTP 로 보내고, 실패하면 재시도하고, 어디까지 읽었는지 오프셋을 기록하고, 프로세스가 죽으면 그 지점부터 다시 시작하고, 부하가 늘면 인스턴스를 늘리고, 인스턴스끼리 Kafka 파티션을 나눠 갖게 하고...

여기서 말하는 파티션은 Kafka 토픽을 여러 줄로 쪼갠 조각입니다. 앞에서 본 DB 청크와는 다른 것입니다. 토픽을 6 개 파티션으로 만들면 메시지가 6 줄로 나뉘어 쌓이고, 읽는 쪽이 여럿이면 그 6 줄을 나눠 가질 수 있습니다. 이것이 Kafka의 병렬 처리 단위입니다.

문제는 이 코드가 어디서나 똑같다는 점입니다. "어디서 읽고 / 어떻게 파싱하고 / 어디로 쓴다" 세 가지만 바뀌고 나머지 기계 장치는 전부 동일합니다. 그래서 그 기계 장치를 프레임워크로 만들고, 바뀌는 세 가지만 설정 파일로 받는 도구가 나왔습니다. 이것이 코드 없이 연결된다는 말의 정확한 의미입니다. 코드가 마법처럼 사라진 게 아니라, 반복되는 부분을 남이 미리 짜 놓은 것입니다.

Kafka Connect 는 Apache Kafka 배포판에 함께 들어 있는 워커(worker) 프로세스입니다. 워커란 여기서 "Kafka Connect 프레임워크 자체를 돌리는 별도의 상주 프로그램"을 말합니다. Kafka 브로커와는 다른 프로세스이고, 보통 여러 대를 띄워 한 무리(클러스터)로 묶습니다.

이 워커 위에 커넥터(connector)라는 플러그인을 엽니다. 커넥터는 JAR 파일로 배포됩니다. JAR(Java ARchive)는 자바로 짠 프로그램과 부속 파일을 하나로 압축해 묶은 파일입니다. 확장자가 .jar 이고, 자바 프로그램의 "설치 파일" 같은 것이라고 보면 됩니다. 이 파일을 지정된 폴더에 넣으면 워커가 읽어 들입니다. Kafka Connect 전체가 JVM 위에서 돌립니다. JVM(Java Virtual Machine, 자바 가상 머신)은 자바 프로그램을 실제로 실행해 주는 엔진입니다. 즉 이 도구를 쓰려면 자바 실행 환경이 필요하고, 메모리도 JVM 몫으로 따로 잡힙니다.

커넥터는 두 종류입니다 — 외부에서 Kafka 로 가져오는 소스 커넥터(Source Connector), Kafka 에서 외부로 내보내는 싱크 커넥터(Sink Connector). 설정은 JSON 한 덩어리를 REST API 로 등록합니다. REST API 란 HTTP 요청(주소 + 본문)으로 프로그램에 명령을 주고받는 창구를 말합니다. 설정 파일을 서버에 올려 두고 재시작하는 게 아니라, 돌고 있는 워커에게 HTTP 로 "이 커넥터를 이 설정으로 띄워 라"라고 말하는 방식입니다.

```
{
  "name": "influxdb-sink",
  "config": {
    "connector.class": "io.confluent.influxdb.InfluxDBSinkConnector",
    "topics": "sensor.telemetry.clean",
    "influxdb.url": "http://influxdb:8086",
    "tasks.max": "3"
  }
}
```

주의. 위 예의 io.confluent.influxdb.InfluxDBSinkConnector 는 Confluent 가 만든 상용 라이선스 커넥터입니다. 형식을 보여 주기에 가장 널리 알려진 예라서 인용했을 뿐, 오픈소스만으로 구성하는 이 프로젝트에서는 쓸 수 없습니다. AR-100 의 의사결정 기록(docs/journal/08-decisions.md ADR-001)이 Kafka Connect 계열을 기각한 사유를 그대로 적어 두고 있습니다 — "Confluent 판은 독점 라이선스. 대안 커넥터는 설정 복잡도가 Telegraf 보다 높음." 같은 판단이 docs/journal/01-analysis.md 의 실사 결과 표에도 있습니다. Kafka Connect 를 검토할 때 는 커넥터 하나하나의 라이선스를 따로 확인해야 한다는 것이 여기서 엄을 교훈입니다. 프레임워크가 아파치 라이선스라고 해서 그 위에 올리는 커넥터가 전부 자유롭게 쓸 수 있는 것이 아닙니다.

Kafka Connect 가 대신 해 주는 것: 오프셋 자동 관리, 장애 시 재시작, tasks.max 만 올리면 병렬 확장, 컨버터를 통한 포맷 변환, 실패한 레코드를 별도 토픽으로 보내는 데드 레터 큐(Dead Letter Queue).

여기서 "포맷 변환"에 나오는 이름들도 풀어 둡니다.

- Avro, Protobuf(Protocol Buffers) — 데이터를 바이너리로 압축해 주고받는 형식입니다. JSON 은 사람이 읽을 수 있는 대신 필드 이름이 매 건마다 반복돼 크기가 큼니다. Avro·Protobuf 는 필드 이름을 빼고 값만 바이너리로 담아 훨씬 작고 빠릅니다. 대신 스키마(어느 자리에 무엇이 들어있는지 적은 설계도)가 따로 있어야 읽을 수 있습니다.

- 스키마 레지스트리(Schema Registry) — 그 스키마들을 한군데 모아 등록·보관하는 서버입니다. 메시지는 스키마 전문 대신 등록 번호만 붙이고, 읽는 쪽이 그 번호로 레지스트리에서 스키마를 받아 옵니다. 스키마가 바뀔 때 옛 데이터가 여전히 읽히는지(호환성)를 검사해 주는 역할도 합니다.

Telegraf 는 InfluxData 가 만든 플러그인 기반 수집 에이전트입니다. 의존성 없는 Go 바이너리 하나이고, TOML 설정 파일 한 장으로 동작합니다. TOML(Tom's Obvious Minimal Language)은 설정 파일 전용 형식입니다. 키 = 값 으로 값을 쓰고, [섹션] 으로 묶고, [[반복 섹션]] 처럼 대괄호를 두 번 쓰면 같은 종류를 여러 개 나열한다는 뜻입니다. 뒤에 나올 [[inputs.kafka_consumer]] 가 네 번 반복되는 것이 바로 이 문법입니다.

구조가 아주 단순합니다 — 입력(inputs) → 처리(processors) → 출력(outputs). 입력 플러그인 200 종 이상, 출력 플러그인 50 종 이상이 하나의 바이너리에 전부 들어 있습니다. 설치할 플러그인을 고르는 게 아니라 설정 파일에서 쓸 플러그인을 켜는 방식입니다.

```
[[inputs.kafka_consumer]]          # 어디서 읽나
  brokers = ["kafka:9092"]
  topics  = ["sensor.telemetry.clean"]
  data_format = "json_v2"          # 어떻게 파싱하나
[[outputs.influxdb_v2]]           # 어디로 쓰나
  urls    = ["http://influxdb:8086"]
  bucket  = "process"
```

둘의 차이. Kafka Connect는 Kafka 생태계 전용이고 JVM 클러스터로 돌아가며, 워커들끼리 작업을 나눠 갖는 자체 조정 계층과 스키마 관리가 강합니다. Telegraf는 Kafka 밖의 세상(MQTT, Modbus, SNMP, 시스템 지표)까지 다루고 훨씬 가볍지만, Telegraf 자체에는 여러 대를 묶어 작업을 배분하는 조정 계층이 없습니다.

다만 이 말을 "Telegraf는 수평 확장이 안 된다"로 읽으면 틀립니다. 입력이 Kafka 라면 확장의 조정자를 Kafka 가 맡아 주기 때문입니다. AR-100의 telegraf/sink.conf는 입력이 전부 consumer_group 을 지정한 kafka_consumer 이고, 읽는 토픽의 Kafka 파티션 수는 각각 6 / 6 / 3 / 3 입니다. Telegraf 컨테이너를 여러 개 띄우면 Kafka 의 컨슈머 그룹 프로토콜이 파티션을 겹치지 않게 자동 분배합니다. 파티션이 6 이면 최대 6 대까지 나눠 받습니다. 차이는 "확장이 되느냐"가 아니라 확장을 누가 조정하느냐입니다 — Kafka Connect는 자기가, Telegraf는 Kafka 가 합니다. 그래서 Kafka 가 아닌 입력(파일, SNMP 등)에서는 이 자동 분배가 없습니다.

둘 다 같은 자리에 올 수 있는 도구이고, 설계 보고서도 Tier 4 를 설명하면서 "Kafka Connect 또는 Telegraf 컨슈머를 통해 InfluxDB 로 배치 단위 고속 적재"라고 두 가지를 나란히 적었습니다.

AR-100에서는, 저장소로 InfluxDB 2.7 을 씁니다(.env : INFLUXDB_VERSION=2.7). 조직(org) 이름은 uengine , 버킷 이름은 process , 외부 노출 포트는 27086 입니다(.env : PORT_INFLUXDB=27086). 컨테이너를 처음 띄울 때 docker-compose.yml 의 환경변수로 초기화됩니다.

```
DOCKER_INFLUXDB_INIT_MODE: setup
DOCKER_INFLUXDB_INIT_ORG: ${INFLUX_ORG}          # uengine
DOCKER_INFLUXDB_INIT_BUCKET: ${INFLUX_BUCKET}    # process
DOCKER_INFLUXDB_INIT_RETENTION: 30d              # ← 보존 정책
DOCKER_INFLUXDB_INIT_ADMIN_TOKEN: ${INFLUX_TOKEN}
```

InfluxDB 2.x 에서 버킷(bucket)은 1.x 의 "데이터베이스 + 보존 정책"을 합친 개념입니다. 버킷을 만들 때 보존 기간을 같이 정하고, 그 기간이 지난 데이터는 엔진이 알아서 지웁니다. 여기서는 30 일 입니다. 별도의 삭제 스크립트가 없는 이유입니다. 그 버킷이 속하는 org 가 uengine 이고, 조회 API 를 호출할 때 이 이름을 함께 넘겨야 합니다.

TimescaleDB 는 검토했지만 채택하지 않았습니다. ARCHITECTURE.md 7 절 미채택 표에 사유가 적혀 있습니다 — "InfluxDB 와 택일. 나노초 정밀도 + Line Protocol 푸시가 본 예제에 적합". 이 문장은 그대로 읽으면 오해가 생깁니다. 이 예제의 실제 샘플링 주기는 .env 의 SCAN_INTERVAL_MS=1000 , 즉 1 초이고 실측 유입은 12 건/초입니다(docs/journal/09-metrics.md 9.3). 나노초 해상도가 필요해서 고른 것이 아닙니다. 실제 이유는 데이터 계약이 그렇게 정의돼 있어서 입니다. ARCHITECTURE.md 3 절의 전 구간 공통 스키마가 ts 를 나노초 정수 (1758000000123456789)로 못 박았고, Telegraf 파서도 timestamp_format = "unix_ns" 로 그것을 그대로 받습니다. 변환 없이 흘러보낼 수 있다는 일관성이 이득이지, 나노초 자체가 이 공정에 필요한 정밀도인 것은 아닙니다.

AR-100 · 직접 열어 보는 최소 동선. 이 장의 내용은 실제로 눌러 보면 30 초 만에 확인됩니다.

- ① 주소 확인. make urls 가 전체 접속 주소를 출력합니다. InfluxDB 는 http://localhost:27086 입니다.
- ② 로그인. 초기 계정은 .env 의 INFLUX_USER / INFLUX_PASSWORD (admin / admin12345)입니다. 이 것은 DOCKER_INFLUXDB_INIT_USERNAME · _PASSWORD 로 컨테이너가 처음 뜰 때 만들어진 계정입니다.
- ③ 버킷 확인. 왼쪽 메뉴의 Load Data → Buckets 에 process 버킷이 보이고, 보존 기간이 30 일로 표시됩니다. 앞에서 본 DOCKER_INFLUXDB_INIT_RETENTION: 30d 가 여기에 나타난 것입니다.
- ④ 시리즈 확인. Data Explorer 에서 버킷 process 를 고르면 measurement 목록(process , process_raw , anomaly , alerts)이 뜨고, 하나를 고르면 그 measurement 가 가진 태그 키와 태그 값이 그대로 나열됩니다. 이 장에서 계산한 시리즈 구성을 눈으로 확인하는 자리입니다.

⑤ API 로 직접 묻기. 화면 없이 확인하려면 HTTP 로 Flux 를 보내면 됩니다. 여기서는 토큰이 필요합니다 — 로그인 계정이 아니라 .env 의 INFLUX_TOKEN 을 Authorization: Token ... 헤더에 넣어야 합니다. 아래는 scripts/verify.py 의 influx_query() 가 실제로 만드는 요청과 같은 형태입니다.

```
curl -s -XPOST "http://localhost:27086/api/v2/query?org=uengine" \
-H "Authorization: Token iiot-scada-demo-token-0000000000" \
-H "Content-Type: application/vnd.flux" \
-H "Accept: application/csv" \
--data 'from(bucket:"process")
|> range(start: -5m)
|> filter(fn: (r) => r._measurement == "process")
|> group(columns: ["tag"])
|> count()'
```

태그별 최근 5분 건수가 CSV 로 돌아옵니다. group(columns: ["quality"]) 로 바꾸면 이 장 뒤에 나오는 실측/보간 분포가 그대로 나옵니다.

토큰에 대한 경고. iiot-scada-demo-token-0000000000 은 예제용으로 .env 에 고정해 둔 값입니다. 로컬에서 실행하려고 일부러 단순하게 만든 것이므로, 실제 설비나 공용 서버에 그대로 옮겨 쓰면 안 됩니다. 토큰 하나로 이 org 의 모든 버킷을 읽고 쓸 수 있습니다.

AR-100 에서는. Kafka 와 InfluxDB 를 잇는 것은 Telegraf 입니다. 컨테이너 이름은 telegraf-sink , 설정 파일은 telegraf/sink.conf 한 장이며, 자바 코드도 파이썬 코드도 없습니다. 이 파일 안에 입력 4 개 + 출력 1 개가 들어 있습니다. Kafka 토픽 4 개를 각각 다른 measurement 로 받아 InfluxDB 버킷 하나에 씁니다.

Kafka 토픽 (파티션)	InfluxDB measurement	컨슈머 그룹	태그	내용
sensor.telemetry.clean (6)	process	telegraf-influx-clean	4 개	중복제거 + 결측보간 완료
sensor.telemetry.raw (6)	process_raw	telegraf-influx-raw	4 개	무손실 원본 (감사 추적 기준선)
sensor.anomaly.score (3)	anomaly	telegraf-influx-score	3 개	재구성 오차 · 추론 지연
sensor.alerts (3)	alerts	telegraf-influx-alerts	6 개	Tier1 규칙 + Tier2 ML 통합 알람

컨슈머 그룹(consumer group)이란. 같은 그룹 이름(group.id)을 쓰는 컨슈머들의 모임입니다. Kafka 는 한 그룹 안에서 토픽 파티션을 겹치지 않게 나눠 주고, "어디까지 읽었는지"(오프셋)도 그룹 단위로 따로 기록합니다. 그래서 그룹이 다르면 같은 토픽을 서로 간섭 없이 각자 처음부터 읽을 수 있습니다.

왜 토픽마다 그룹을 따로 뒀는가. 두 가지 이유입니다. 첫째, 네 입력이 서로 다른 토픽을 구독하는 데 그룹을 공유하면 같은 그룹 구성원들이 서로 다른 토픽 목록을 들고 오게 되어 파티션 배분이 충돌합니다. 둘째, 그룹을 나눠 두면 컨슈머 랙이 그룹 × 토픽 단위로 따로 보입니다. 앞에서 본 경보 규칙 sum by (consumergroup, topic) (kafka_consumergroup_lag) 이 이 구조를 전제로 쓰였습니다. 알람 적재가 밀리는지 원본 적재가 밀리는지를 따로 볼 수 있어야 대응이 됩니다.

process measurement 의 파서 설정이 이 장의 모든 개념이 만나는 자리입니다.

```
[[inputs.kafka_consumer.json_v2]]
  measurement_name = "process"
  timestamp_path    = "ts"
  timestamp_format  = "unix_ns"      # ← 데이터 계약의 나노초를 그대로 받는다
[[inputs.kafka_consumer.json_v2.tag]]
  path = "site"                      # ㄱ
[[inputs.kafka_consumer.json_v2.tag]]
  path = "device"                    # | 태그 = 색인됨 = measurement 와 함께 시리즈를 결정
[[inputs.kafka_consumer.json_v2.tag]]
  path = "tag"                       # |
[[inputs.kafka_consumer.json_v2.tag]]
  path = "quality"                   # ㄴ
[[inputs.kafka_consumer.json_v2.field]]
  path = "value"                     # 필드 = 색인 안 됨 = 그냥 기록
  type = "float"
```

태그 4 개, 필드 1 개. 이 설정이 InfluxDB 안에서 만들어 내는 라인 프로토콜 한 줄은 다음 형태입니다(설정과 ARCHITECTURE.md 3 절 데이터 계약에서 도출한 형태입니다).

```
process,site=AR-100,device=reactor-line-01,tag=TT-101,quality=GOOD value=72.43 17580000000123456789
```

이 줄에서 맨 앞 process 부터 마지막 태그 quality=GOOD 까지가 시리즈 키이고, value=72.43 이 필드, 마지막 숫자가 나노초 타임스탬프입니다.

쓰기 설정은 [agent] 절에 있습니다 — interval = "1s" , metric_batch_size = 5000 , metric_buffer_limit = 200000 , flush_interval = "2s" . 2 초마다 최대 5,000 건씩 묶어 보내고, InfluxDB 가 잠시 응답하지 않으면 메모리에 20 만 건까지 물고 기다립니다. 이것이 설계 보고서가 말한 "비동기 벌크 쓰기"의 실제 설정값입니다.

AR-100 · 카디널리티를 measurement 별로 세어 본다. 앞에서 배운 대로 measurement 안에서는 곱하고, measurement 사이에서는 더합니다.

measurement	태그	계산	시리즈
process	site · device · tag · quality	$1 \times 1 \times 12 \times 3$	최대 36
process_raw	site · device · tag · quality	$1 \times 1 \times 12 \times 1$	12
anomaly	site · device · top_contributors	$1 \times 1 \times (\text{값 가짓수})$	가변 — 아래 주의 참조
alerts	site · device · tag · alert_type · severity · detector	$1 \times 1 \times 12 \times 5 \times 2 \times 4$	이론상 최대 480

"36 시리즈"는 이 데이터베이스 전체가 아니라 process 하나의 값입니다. 전체는 이 네 줄을 더한 값입니다.

· 태그 12 종은 LT-101 , LT-102 , TT-101 , TT-102 , PT-101 , FT-101 , FT-102 , IT-101 , IT-102 , VT-101 , pH-101 , CT-101 입니다.

· quality 는 코드가 실제로 만드는 값이 3 종입니다 — GOOD , INTERPOLATED_LOCF , INTERPOLATED_LINEAR (Reading.java 13 행, Interpolator.java 75 행). ARCHITECTURE.md 3 절은 여기에 STALE 을 더해 4 종으로 적고 있으나 STALE 을 만들어 내는 코드는 저장소에 없습니다. 이 책은 3 으로 씁니다. process_raw 는 보간을 하지 않으므로 quality 가 GOOD 하나뿐이고, 따라서 12 시리즈 입니다. 브리지가 유효한 값에만 quality = "GOOD" 을 붙이는 것이 telegraf/bridge-edgex.conf 에 그대로 있습니다.

· alerts 의 480 은 이론상 상한입니다. 실제로는 조합이 자유롭지 않습니다 — flink/sql/01_sources.sql 의 주석대로 detector 가 TIER1_RULE 이면 alert_type 은 THRESHOLD_USL / THRESHOLD_LSL 이고 severity 는 CRITICAL 로 고정되는 식이라, 실제로 생기는 조합은 훨씬 적습니다. 그래도 상한을 세어 두는 습관이 중요합니다. 태그를 하나 더 붙이자면 제안이 들어 왔을 때 곱이 어떻게 변하는지 그 자리에서 답할 수 있어야 하기 때문입니다.

· 센서 일련번호나 배치 ID 를 태그로 붙이지 않았기 때문에 어느 measurement 에서도 곱이 커지지 않습니다.

이 규모에서 InfluxDB 컨테이너는 메모리 106 MiB, CPU 0.2%를 씁니다(12 건/초 정상 운전, docs/journal/09-metrics.md 9.2 실측). 전체 스택 29 개 컨테이너 합계 약 9.1 GiB 중 1% 남짓입니다. 시계열 DB 는 잘 설계하면 파이프라인에서 가장 가벼운 축에 속합니다. 무거운 것은 Flink(3.35 GiB)와 Kafka(2.17 GiB)입니다.

AR-100 에서는. 보존 기간이 계층마다 다르게 설정돼 3 층 구조를 이룹니다.

계층	보존 기간	설정 위치	이유
Kafka sensor.telemetry.raw /clean / anomaly.score	24 시간 (86400000 ms)	kafka/create-topics.sh	Kafka 는 버퍼 이지 저장소 가 아니다. 하루면 재처리 에 충분
Kafka sensor.alerts	7 일 (604800000 ms)	kafka/create-topics.sh	알람은 사후 확인 기간이 길다
InfluxDB 버킷 process	30 일	docker-compose.yml	히스토리안. 공정 이력 조회 대상
Prometheus	7 일 (--storage.tsdb.retention.time=7d)	docker-compose.yml	인프라 메트릭. 장기 보관 가치 없음

저장량의 대가를 같이 적어 둡니다. 이 구성은 같은 텔레메트리를 process 와 process_raw 두 measurement 에 이중으로 적재합니다. 정제본과 원본을 둘 다 남기기로 한 선택의 직접적인 비용 이고, InfluxDB 저장량이 약 2 배가 됩니다. 30 일 보존이므로 30 일치를 두 벌 들고 있는 셈입니다. docs/journal/08-decisions.md 의 ADR-004 도 이 대가를 명시해 뒀습니다 — "저장 비용은 2 배지만 시계열 압축으로 실질 부담이 작음." 앞에서 본 압축 덕분에 감당할 만하다는 판단이지, 비용이 없다는 뜻이 아닙니다. 12 건/초 규모라 문제가 되지 않을 뿐, 설비 수천 대로 올리면 이 2 배가 먼저 아픕니다.

Kafka 토픽에는 compression.type=lz4 도 걸려 있습니다. lz4 는 압축률보다 속도를 우선하는 압축 알고리즘입니다. gzip 보다 압축률은 낮지만 압축·해제에 쓰는 CPU 가 훨씬 적습니다. Kafka 브로커 는 메시지를 한 건씩이 아니라 묶음(배치) 단위로 압축하는데, 초당 수천~수만 건이 지나가는 길목에서는 몇 퍼센트 더 줄이는 것보다 지연을 늘리지 않는 것이 중요합니다. 그래서 스트리밍 경로의 기본 선택이 lz4 입니다. 결과적으로 이 파이프라인에서는 전송 계층(Kafka lz4)과 저장 계층(InfluxDB 델타-오브-델타 + XOR)에서 각각 다른 방식의 압축이 일어납니다.

다운샘플링은 이 예제에서 저장 시점이 아니라 조회 시점에 일어납니다. 연속 집계나 InfluxDB 태스크로 요약본을 따로 만들어 두지 않았습니. 대신 Grafana 대시보드 grafana/dashboards/01-process.json 의 Flux 쿼리 마지막에 aggregateWindow 가 붙어 있습니다.

```
from(bucket: "${bucket}")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "process")
  |> filter(fn: (r) => r._field == "value")
  |> filter(fn: (r) => r["tag"] == "TT-101" or r["tag"] == "TT-102")

|> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
|> keep(columns: ["_time", "_value", "tag"])
```

v.windowPeriod 는 Grafana 가 화면 폭과 선택한 시간 범위에 맞춰 자동 계산하는 값입니다. 1 시간 을 보면 몇 초 단위, 7 일 을 보면 몇 분 단위로 알아서 묶입니다. 원본은 1 초 간격 그대로 남아 있고, 화면에 그릴 때만 평균을 냅니다. 브라우저에 300 만 점을 보내는 대신 800 점을 보내는 방식입니다.

AR-100에서는. quality 를 태그로 저장한 선택이 감사 추적(Audit Trail)을 성립시킵니다. 이것이 이 장의 핵심 결과입니다.

설계 보고서는 이렇게 경고합니다 — "원천 데이터 레벨에서 인위적으로 계산된 가공치를 덮어쓰게 되면 실제 물리 계측치와 추정치 간의 경계가 모호해져 향후 사고 발생 시 감사 추적의 법적·기술적 신뢰성을 훼손할 수 있다."

AR-100 은 이 경고를 구조로 지킵니다. 원본은 process_raw 에 결측 구간을 공백인 채로 남기고, 보간값은 process 에만 넣되 quality 태그에 보간 방식을 적습니다. 태그는 색인되므로 "보간된 값 만 골라 보기"가 조회 한 줄로 됩니다.

quality 의 구현 정보는 2 권 5 장의 세 값입니다. 이 원문 위치는 STALE 를 네 번째 값으로 다시 소개하지만, 34 쪽의 "STALE 생성 코드 없음"과 다르므로 구현 완료로 해석하지 않습니다. [원문 34, 259 쪽]

실측 분포는 이렇습니다(docs/journal/09-metrics.md 9.6).

quality	건수	비율	의미
GOOD	15,858	99.74 %	센서가 실제로 찍은 값
INTERPOLATED_LINEAR	42	0.26 %	Flink 가 선형 보간으로 채운 추정치

정제본과 원본의 건수를 비교할 때 주의할 것. 같은 실측에서 최근 5 분 기준 measurement 별 적재 건수도 측정됐습니다 — process 3,576 건 / process_raw 3,534 건 / anomaly 1,196 건 / alerts 426 건(docs/journal/06-tier4-scada.md). 여기서 42 건 차이를 "보간으로 늘어난 만큼"이라고 설명 하면 틀립니다. 두 가지 이유가 있습니다.

① clean 은 보간만 하는 게 아니라 중복도 지웁니다. kafka/create-topics.sh 주석부터가 "중복제거 + 결측보간 완료"라고 적어 뒀고, 실제 코드가 그렇습니다. Interpolator.java 는 태그별로 마지막에 내보낸 시각을 상태로 들고 있다가, 그보다 같거나 이른 시각이 다시 오면 버립니다.

그러니 정제본 건수는 보간으로 늘 수도, 중복 제거로 줄 수도 있습니다. 한 방향이 보장되지 않습니다.

② 두 숫자는 같은 창을 켜 값이 아닙니다. process 와 process_raw 는 telegraf/sink.conf 에서 서로 다른 컨슈머 그룹으로, 각각 offset = "newest" 부터 읽기 시작합니다. 즉 두 적재의 시작 시점이 같다는 보장이 없습니다. "최근 5 분"이라는 창 안에서 한쪽이 조금 일찍 시작했다면 그만큼 건수가 다릅니다.

실제로 같은 창을 제대로 켜 값은 따로 있습니다. 저널 9.3의 처리량 실측은 telemetry.raw → clean 이 40 초 동안 528 → 528, 손실 0 입니다. 두 계열은 같습니다. 3,576 과 3,534 라는 숫자로 말할 수 있는 것은 두 계열이 비슷한 규모로 나란히 쌓이고 있다는 것까지입니다.

보간이 실제로 일어나는 장면은 따로 측정돼 있습니다. dropout 고장을 TT-101 에 15 초 주입하면 — raw 토픽에는 15.0 초 공백이 그대로 남고(대조군 LT-102 는 연속), clean 토픽에는 14 건이 보간 으로 채워지며 값이 70.132 에서 70.204 로 균등하게 증가합니다(저널 9.6). 원본은 비어 있고 정제본만 채워지는 것이 여기서 눈에 보입니다.

Grafana 에는 이 구분을 그대로 보여 주는 패널이 있습니다.

```
|> group(columns: ["quality"])
|> aggregateWindow(every: v.windowPeriod, fn: count, createEmpty: true)
```

여기서 createEmpty 가 다른 이유가 중요합니다. 앞의 온도 트렌드 패널은 createEmpty: false 였는데 이 품질 패널은 true 입니다. 실수가 아니라 의도입니다.

· 트렌드 패널(false) — 값이 없는 구간에 빈 행을 만들면 그 창 의 평균이 없다고 처리되어 선이 0 으로 떨어지거나 이상하게 이어집니다. 온도가 0 °C 로 곤두박질친 것처럼 보이면 안 됩니다. 데이터가 없으면 선을 끊어서 "그때는 값이 없었다"를 그대로 보여 주는 것이 좋습니다.

· 품질 패널(true) — 여기 집계 함수는 count , 즉 개수 세기입니다. 보간이 한 건도 없었던 구간은 "데이터 없음"이 아니라 "0 건"이 진실입니다. false 로 두면 보간이 0 건인 구간의 선이 통째로 사라져서 "보간이 없었던 시간"과 "측정하지 않은 시간"을 구별할 수 없게 됩니다. 같은 옵션이 패널의 의미에 따라 정반대로 설정되는 자리입니다.

사고가 났을 때 "이 시각의 온도 72.4 도는 센서가 실제로 켜 값인가, 아니면 시스템이 추정 한 값인가"라는 질문에 히스토리안이 답할 수 있습니다. 이것이 감사 추적이 성립한다는 말의 뜻입니다. 그리고 앞에서 본 설계 보고서의 판단 — 모니터링만 할 거면 파이프라인에서 보간하지 말고 조회 시점에 DB 함수로 메워라 — 을 이 프로젝트가 따르지 않은 이유도 분명합니다. 오토인코더에 완전한 텐서를 넣어야 하므로 파이프라인 안에서 보간이 필요하고, 대신 원본 토픽 분리 + quality 태그로 설계 보고서가 걱정한 감사 추적을 지킨 것입니다.

주의. 이 예제 안에도 카디널리티가 자랄 수 있는 자리가 하나 있습니다. anomaly measurement 의 top_contributors 가 태그로 설정돼 있는데(telegraf/sink.conf), 이 값은 Flink 가 String.format("%s(%.0f%%)", ...) 로 만드는 문자열입니다. 실측 예는 CT-101(52%), pH-101(30%) 처럼 백분율이 매번 달라지는 형태입니다. 즉 레코드마다 새 시리즈가 생길 수 있는 구조이고, 앞의 경고 목록에 있던 "백분율이 섞인 설명 문자열"에 정확히 해당합니다. 게다가 이 값에는 쉼표와 괄호가 들어 있어 라인 프로토콜 이스케이프 대상이기도 합니다. 그리고 이 문제는 이미 드러나 있습니다. 원문 작성자의 PC 에서 직접 세어 보니 30 분 동안 서로 다른 태그 값이 14,819 개 생겼고, 그 결과 Grafana 의 ML 대시보드에서 숫자 칸 패널 두 개가 값 1,748 개를 한 칸에 겹쳐 그려 읽을 수 없는 상태입니다. 화면 사진과 조회 근거는 2 권 원문 4.2 절에 있습니다. 설비 수천 대 규모로 올리기 전에도 이미 카디널리티 폭발의 전형입니다. 이 항목은 필드로 옮기는 것이 맞습니다 — 화면에서 읽기만 할 값이지 그룹으로 묶어 조회할 값이 아니기 때문입니다. InfluxDB 를 쓰고 있다는 사실이 이 문제를 면제해 주지 않습니다.

② "일단 다 넣고 나중에 정리하자" — 시계열 DB 에서 가장 위험한 생각입니다. 카디널리티는 한 번 올라가면 내리기 어렵습니다. 이미 만들어진 시리즈 색인은 데이터를 지워도 한동안 남고, 지우는 작업 자체가 부하입니다. 태그 설계는 처음에 하는 것입니다.

같은 자리의 다른 도구들 InfluxDB — 시계열 전용 엔진. 나노초 타임스탬프, Line Protocol 푸시, measurement + 태그 색인. 순수 텔레메트리(원격 계측 데이터)를 푸시로 부어 넣는 데 가장 직접적입니다. 대신 조인이 없고, 2.x 의 Flux 는 SQL 과 문법이 달라 학습이 필요하며, 1.x·2.x·3.x 의 질의 언어와 저장 엔진이 계속 바뀌어 왔다는 점도 도입 시 고려할 사항입니다. AR-100 의 선택(2.7). TimescaleDB — PostgreSQL 확장. 하이퍼테이블로 시간 분할, 연속 집계로 다운샘플링, add_compression_policy 로 컬럼 단위 압축(문서상 10~20 배), 그러면서 SQL 과 조인이 그대로 남습니다. 설비 마스터·작업지시·품질검사 같은 마스터 데이터와 센서를 한 쿼리로 묶어야 한다면 이쪽이 유리합니다. 타임스탬프는 마이크로초까지입니다. PDF 본문도 "InfluxDB(또는 대규모 분석 워크로드에 최적화된 TimescaleDB)"라고 나란히 놓으며, 오히려 대규모 분석 쪽에 이름을 올립니다. 대가는 PostgreSQL 의 운영 특성(MVCC 팅창과 VACUUM , 커넥션 관리)을 그대로 물려받는다는 점입니다. Prometheus — 같은 자리에 올 수 없는 도구. 목적이 인프라 감시이고, 수집이 Pull, 타입이 Float64 하나, 시리즈 색인을 전부 메모리에 올려 카디널리티에 극도로 취약합니다. 설계 보고서는 센서 영구 저장소로 쓰는 것을 "불가능"이라고 단정하고 "순수 파이프라인 인프라 가용성 감시 계층으로 완전히 격리"하라고 결론 냅니다. AR-100 은 이 격리를 자동 검사로 강제합니다. 일반 PostgreSQL / MySQL — 확장 없이 그대로. 초당 수십 건 규모의 소규모 설비라면 실제로 충분합니다. 다만 억 단위 행에서 색인이 메모리를 넘어 랜덤 I/O 가 나고, 오래된 데이터 삭제가 몇 시간짜리 작업 이 되며, 시리즈 단위 압축의 이점을 못 얻습니다. 규모가 예상되면

처음부터 TimescaleDB를 켜 두는 편이 낫습니다 — 같은 PostgreSQL 이므로 전환 비용이 작습니다. 상용 히스토리안 (OSIsoft PI System 등) — 공정 산업에서 수십 년간 쓰인 전통적 선택지. 산업 프로토콜 연결과 공정 엔지니어용 도구가 완비돼 있지만 상용 라이선스이고, Kafka·Flink 같은 현대 스트리밍 스택과 의 연동은 별도 어댑터가 필요합니다. AR-100 이 오픈소스만으로 구성하는 것과 대비되는 축입니다. 연결 도구 — Kafka Connect — Kafka 생태계의 표준 연결 프레임워크. 커넥터 JAR(자바 프로그램 묶음 파일) + JSON 설정을 REST API 로 등록합니다. 오프셋 관리·재시작·tasks, max 병렬 확장·데드 레터 큐를 프레임워크가 제공하고, 워커들끼리 작업을 나눠 갖는 조정 계층을 자기가 갖고 있습니다. Kafka 를 중심에 둔 대규모 구성에 적합합니다. 이 프로젝트가 쓰지 않은 이유는 성능이 아니라 라이선스와 복잡도입니다 (ADR-001: Confluent 판 커넥터는 독점 라이선스, 대안 커넥터는 설정 복잡도가 Telegraf 보다 높음). 연결 도구 — Telegraf — 단일 Go 바이너리 + TOML 설정. 입력/처리/출력 플러그인이 전부 내장돼 있어 설정으로 켜기만 하면 됩니다. Kafka 밖(MQTT, Modbus, SNMP)까지 다루므로 AR-100 은 MQTT→Kafka 브리지와 Kafka→InfluxDB 싱크에 같은 도구를 씁니다. Telegraf 자체에는 여러 대를 묶어 작업을 배분하는 조정 계층이 없다는 점이 Kafka Connect 와의 차이입니다 — 다만 이 장의 구성처럼 입력이 kafka_consumer 라면 컨슈머 그룹이 파티션을 자동 분배하므로 컨테이너를 늘려 수평 확장할 수 있습니다. 조정자가 Telegraf 가 아니라 Kafka 일 뿐입니다.

9 SCADA 와 관제 — 보는 것과 움직이는 것

원문 교본 263-290 쪽의 설명을 통한

SCADA 화면은 현장 값을 읽는 방향과 설정·명령을 쓰는 방향을 함께 다룹니다. AR-100에서는 FUXA가 Modbus를 직접 읽고 쓰며, MQTT로 받는 것은 공정 알람입니다. Grafana는 이력과 분석 추세를, Prometheus와 Alertmanager는 파이프라인의 상태를 말합니다.

SCADA라는 말을 글자 하나씩 뜯어봅니다

SCADA는 Supervisory Control And Data Acquisition의 머리글자입니다. 우리말로로는 감시 제어 및 데이터 수집이라고 옮깁니다. 이름이 길고 딱딱해서 그냥 "공장 모니터링 화면" 정도로 뭉뚱그려 이해하는 경우가 많은데, 그러면 이 장 전체를 오해하게 됩니다. 이름 안에 서로 다른 두 가지 일이 들어 있기 때문입니다.

Data Acquisition(데이터 수집)은 현장에 흩어진 계측기에서 값을 주기적으로 읽어 오는 일입니다. 온도 72.86 °C, 압력 3.22 barg 같은 숫자를 일정한 간격으로 읽어 와서 화면에 띄우고 기록합니다. 이건 우리가 이미 아는 일입니다.

Supervisory Control(감시 제어)은 그 반대 방향입니다. 사람이 화면에서 버튼을 누르면 그 명령이 현장 기기까지 내려가 물리적인 변화를 일으킵니다. 펌프가 멈추고, 밸브가 열리고, 히터가 켜집니다. "Supervisory(감독의, 상위의)"라는 단어가 붙은 이유는 이 제어가 직접 제어가 아니기 때문입니다. 펌프를 초당 수십 번 계산해 가며 돌리는 것은 현장의 PLC(Programmable Logic Controller, 프로그램 가능 논리 제어기)나 전용 제어기가 합니다. SCADA는 그 위에서 "펌프를 켜라", "속도 목표를 85 %로 해라" 같은 상위 지시만 내립니다. 초 단위 지시를 내리는 감독자이지, 밀리초 단위로 손을 놀리는 작업자가 아닙니다.

왜 이런 물건이 생겼는가. 1960년대 이전의 공장 제어실은 벽 한 면이 통째로 계기판이었습니다. 아날로그 지시계 수백 개가 박혀 있고, 그 옆에 스위치와 레버가 물리적으로 배선되어 있었습니다. 온도를 하나를 추가하려면 현장에서 제어실까지 구리선을 새로 깔아야 했습니다. 계측점이 늘어날수록 배선 다발이 굵어지고, 공장이 넓어질수록(정유소, 상수도관망, 송전선로) 배선 자체가 불가능해졌습니다. 그래서 두 가지 발상이 나왔습니다. 첫째, 현장에 RTU(Remote Terminal Unit, 원격 단말 장치)나 PLC를 두어 근처 계측기들을 모아 디지털 숫자로 바꾸게 합니다. 둘째, 제어실과 현장 사이는 굵은 배선 다발 대신 통신선 한 가닥으로 연결하고 그 위에 통신 규약(프로토콜)을 태웁니다. 이때 쓰인 초기 규약 중 하나가 우리가 3장에서 본 Modbus입니다. 이렇게 하면 계측점이 100 개든 1,000 개든 통신선은 그대로이고, 계기판은 유리 화면 위의 그림으로 바뀝니다. 이 구조 전체에 붙은 이름이 SCADA입니다.

구성요소는 네 덩어리입니다.

필드 기기(Field Device) — 센서(트랜스미터)와 구동기(액추에이터). 온도 트랜스미터, 유량계, 펌프, 제어밸브. 우리 시스템에서는 시뮬레이터가 이 역할을 합니다. 현장 제어기(PLC / RTU) — 필드 기기를 모아 디지털 레지스터로 노출하고, 안전 논리를 결정론적으로 수행합니다. 통신 계층 — Modbus/TCP, OPC-UA 같은 산업용 프로토콜. 값을 읽고(read) 값을 쓰는(write) 두 방향이 모두 뚫려 있습니다. 마스터국(Master Station) — 사람이 보는 쪽. 여기에 HMI와 히스토리안이 있습니다.

OPC-UA는 여기서 처음 나오는 이름이니 풀어 둡니다. OPC Unified Architecture(개방형 플랫폼 통신 통합 아키텍처)의 줄임말로, Modbus보다 한 세대 뒤에 나온 산업용 통신 표준입니다. Modbus는 "40005번 칸에 든 숫자를 주세요"라고 말하는 규약입니다. 그 칸이 온도인지 압력인지, 단위가 무엇인지는 규약이 모릅니다. 사람이 문서로 따로 관리해야 합니다(우리가 plant.yaml에 주소표를 적어 둔 이유가 그것입니다). OPC-UA는 여기서 한 걸음 더 나아가, 기기가 "나는 반응기 R-101이고, 그 아래 TT-101이라는 온도 항목이 있고, 단위는 섭씨이며, 지금 값은 72.86입니다"라고 이름과 구조와 자료형을 함께 알려 줍니다. 이런 성질을 자기 서술적(self-describing)이라고 부릅니다. 여기에 암호화와 인증, 구독 기능이 표준에 포함되어 있습니다. 대신 규약이 무겁고 기기가 비쌉니다. 그래서 현장에는 아직도 Modbus 기기가 압도적으로 많고, 신규 설비나 상위 연결에 OPC-UA가 들어오는 형태로 섞여 있습니다. 두 규약은 경쟁 관계라기보다 같은 자리에서 세대가 다른 선택지입니다. Modbus의 자세한 동작은 3장에서 다뤘습니다.

HMI(Human-Machine Interface, 인간-기계 인터페이스)는 사람과 설비 사이에 놓인 화면 자체를 가리킵니다. SCADA가 시스템 전체의 이름이라면 HMI는 그중 사람이 손대는 표면입니다. 두 단어가 거의 붙어 다녀서 "SCADA/HMI"라고 한 덩어리로 쓰기도 합니다. 초보가 가장 많이 헷갈리는 지점인데, 정리하면 이렇습니다. SCADA는 시스템, HMI는 그 시스템의 화면입니다.

히스토리안(Historian)은 그 옆에 놓인 기록 보관소입니다. HMI가 "지금"을 보여 준다면, 히스토리안은 "지나간 모든 순간"을 담습니다. 이름 그대로 공정의 역사를 적는 물건입니다. 정확히는 프로세스 히스토리안(Process Historian, 공정 이력 저장소)이라고 부르며, 설계 보고서는 이것을 "Flink 나 Kafka Connect를 통해 실시간 유입되는 공정 센서 데이터를 초당 수십만 건 이상 안전하게 적재하는" 역할로 정의합니다. 일반 데이터베이스와 뭐가 다른가 하면, 다루는 데이터의 모양이 다릅니다. 히스토리안에 들어오는 것은 (시각, 태그 이름, 값, 품질)이라는 똑같은 모양의 기록이 초당 수만~수십만 건씩 끝없이 밀려드는 형태입니다. 사람 이름과 주소를 담는 일반 데이터베이스와는 쓰기 패턴 자체가 다릅니다. 그래서 히스토리안은 ① 시간 순서로 들어오는 대량 쓰기에 최적화되고, ② 태그 이름으로 빠르게 뽑아낼 수 있게 색인하고, ③ 오래된 값을 1분 평균·1시간 평균으로 줄여 보관하는 다운샘플링 기능을 갖추고, ④ 수년 단위 장기 보존을 전제로 설계됩니다. 이 네 가지를 갖춘 제품을 히스토리안이라고 부릅니다. 상용 제품으로는 OSIsoft PI System이 대표적이고, 우리 스택에서는 InfluxDB가 이 자리입니다(8장). 즉 SCADA 화면은 지금을 보고, 히스토리안은 과거를 담는다가 이 장 내내 쓰이는 판단 기준입니다.

P&ID(Piping and Instrumentation Diagram, 배관계장도)는 공정 설비를 배관과 계측기 기호로 그린 공식 도면입니다. 건물의 설계 도면 같은 것인데, 대상이 건물이 아니라 공정입니다. 어떤 탱크에서 어떤 배관을 거쳐 어느 펌프로 가고, 그 배관 어디에 어떤 계측기가 붙어 있는지를 표준 기호로 표현합니다. 기호와 태그 이름 규칙은 국제 표준을 따릅니다 — 여기서 말하는 ISA는 국제 자동화 학회(International Society of Automation)로, 계장 기호와 태그 표기를 정한 ISA-5.1 계열 표준을 펴낸 단체입니다. 이 표기를 따르므로 처음 공장에 온 엔지니어도 도면만 보면 배관 구조를 읽을 수 있습니다. TT-101은 "Temperature Transmitter(온도 트랜스미터) 101번", FT-101은 "Flow Transmitter(유량 트랜스미터) 101번"입니다. 앞 글자가 측정 대상, 뒤 글자 T가 전송기를 뜻합니다. 이 이름 규칙이 바로 우리가 계속 써 온 태그(Tag)입니다.

미믹 다이어그램(Mimic Diagram)은 그 P&ID 도면을 화면에 옮겨 놓고, 도면 위의 각 기호가 실시간 값에 따라 살아 움직이게 만든 것입니다. 'mimic'은 흉내낸다는 뜻입니다. 실제 공장의 모습을 화면이 흉내내는 것이죠. 펌프 전류가 0이면 펌프 기호의 색이 죽고, 정상 범위면 살아나고, 과부하 구간이면 경고색으로 바뀝니다. 압력이 한계를 넘으면 해당 계측기 숫자가 붉게 바뀝니다. 운전원은 숫자를 하나씩 읽지 않고 도면 전체의 색깔 패턴으로 공장 상태를 파악합니다. 이것이 꺾은선 그래프 대시보드와 결정적으로 다른 점입니다.

태그 바인딩(Tag Binding)은 그 사이를 잇는 접착제입니다. 화면 위의 어떤 도형이 어느 태그의 값을 보여줄지, 그리고 어떤 버튼이 어느 주소에 무슨 값을 쓸지를 연결해 두는 설정입니다. "이 타원은 IT-101(펌프 전류)을 보고, 값이 0.5 미만이면 한 색, 0.5~12이면 다른 색, 12 초과면 또 다른 색" 같은 규칙과, "이 버튼을 클릭하면 코일 0번에 1을 써라" 같은 규칙이 모두 바인딩입니다. 화면을 만드는 작업의 대부분은 사실 그림 그리기가 아니라 이 바인딩을 빠짐없이 정확히 매는 일입니다.

주의 — 미믹의 색은 만국 공통이 아닙니다. 교과서와 인터넷 글에서 "펌프가 돌면 초록, 멈추면 회색, 밸브가 닫히면 빨강"이라는 설명을 자주 보게 되는데, 이것은 업계 공통 규칙이 아니라 그 화면을 만든 사람이 고른 색입니다. 실제로 발전·석유화학 계열에는 적색을 운전 중/개방으로 쓰는 오래된 관행이 있어, 다른 공장에서 온 사람이 화면을 정확히 거꾸로 읽는 사고가 일어납니다. 게다가 HMI 설계 표준인 ISA-101 계열의 고성능 HMI 지침은 한 걸음 더 나아가 정상 상태는 무채색(회색)으로 두고, 색은 이상 상태에만 쓰라고 권합니다. 화면이 알록달록하면 정작 진짜 이상이 눈에 안 띄기 때문입니다. 그러므로 색 규칙은 언제나 "우리 화면에서는"이라는 한정을 붙여 읽어야 합니다. AR-100 화면이 실제로 쓰는 색은 뒤의 실측 항목에 그대로 적었습니다.

초보가 반드시 헷갈리는 세 지점

첫째, "화면에 버튼이 있으면 SCADA"가 아닙니다. 버튼이 눌렸을 때 그 명령이 산업용 프로토콜로 현장 레지스터까지 내려가야 SCADA입니다. 버튼이 웹 API를 호출해서 어딘가 데이터베이스 값을 바꾸는 것은 SCADA 제어가 아닙니다.

둘째, SCADA가 안전을 책임지는 게 아닙니다. 뒤에 나올 인터록(Interlock)은 SCADA가 아니라 현장 제어기가 수행합니다. 화면이 멈추거나 네트워크가 끊겨도 안전 논리는 동작해야 하기 때문입니다.

그런데 왜 현장 제어기만 그 일을 맡을 수 있는가. 이 장의 핵심 주장이므로 근거를 분명히 해 둡니다. 전용 제어기는 스캔(scan)이라는 방식으로 돕니다. 스캔은 ① 입력을 전부 읽고 → ② 논리를 한 번 계산하고 → ③ 출력을 전부 쓰는 한 바퀴를 말하며, 제어기는 이 한 바퀴를 고정된 주기로 영원히 반복합니다. 주기가 10ms로 설정되면 그 제어기는 10ms마다 반드시 한 바퀴를 끝냅니다. 중간에 다른 프로그램이 끼어들지 않고, 메모리를 정리하느라 멈추는 일도 없고, 화면을 그리느라 늦어지는 일도 없습니다. 그래서 "압력이 6.5를 넘으면 다음 스캔 안에 펌프를 끈다"는 약속을 시간으로 보장할 수 있습니다.

웹 스택은 이 보장을 할 수 없습니다. 브라우저의 자바스크립트는 다른 탭·확장 프로그램과 CPU 를 나눠 쓰고, 서버의 자바 프로그램은 JVM GC(Garbage Collection, 쓰레기 수집)가 갑자기 수십~수백 밀리초 동안 프로그램을 멈춰 세울 수 있습니다. 운영체제는 언제든지 다른 프로세스에 CPU 를 넘겨줍니다. 네트워크는 패킷을 재전송하느라 늦어지고, 사용자의 무선랜은 끊기기도 합니다. 각 구간은 평소엔 빠릅니다. 문제는 "평소엔 빠르다"가 안전 보장이 아니라는 점입니다. 그래서 인터록은 언제나 현장에 남고, SCADA 는 그 결과를 표시만 합니다.

셋째, SCADA 는 데이터 분석 도구가 아닙니다. SCADA 화면은 "지금 이 순간"을 봅니다. 지난 3 개월의 추세를 비교하고 통계를 내는 일은 앞에서 정의한 히스토리안과 분석 대시보드(우리 경우 InfluxDB + Grafana)의 몫입니다. 이 둘을 한 도구로 합치려다 망가지는 사례가 워낙 흔해서, 설계 보고서가 결론 다섯 번째 항목을 통째로 이 분리해 할애했습니다.

Data Acquisition — 감시: 현장 값을 주기적으로 읽어 화면으로



원문 도해 · SCADA 의 관측 방향과 제어 방향 (원문 268 쪽)

SCADA 의 두 방향. 이름 안의 DA 와 SC 가 각각 한 방향씩 담당합니다. 안전 논리는 두 번째 상자 안에서만 돌고, 오른쪽 끝까지 올라오지 않습니다.

Grafana 는 SCADA 가 아닙니다 이 절은 설계 보고서가 11 쪽에서 가장 강하게 주장한 대목이고, 우리 시스템 전체의 설계 근거이기도 하므로 논거를 하나도 빠지 않고 옮깁니다. 오해를 막기 위해 먼저 분명히 해 둡니다. 설계 보고서는 Grafana 를 나쁜 도구라고 말하지 않습니다. 원문 표현 그대로 "시계열 데이터베이스를 조회하여 다차원 메트릭을 동적으로 표출하고 트렌드를 분석하는 시각화 도구로서는 최고의 성능을 발휘한다"입니다. 문제는 Grafana 만 단독 배치해 놓고 그것을 "완전한 SCADA 시스템"이라고 부르는 것이며, 설계 보고서는 이를 "중대한 아키텍처적 결함"이라고 규정합니다.

주의. 설계 보고서가 든 근거는 네 가지입니다. 하나씩 뜯어봅니다.

- ① 구조 자체가 단방향 읽기 전용(Read-only)입니다. Grafana 는 시계열 저장소의 데이터를 읽어 오는 대시보드입니다. 버튼을 눌러 PLC 의 코일 레지스터를 조작하거나 밸브 개도율을 바꾸는 양방향 명령 하달(Write-back) 기능이 네이티브로 제공되지 않습니다. 여기서 네이티브(native)란 플러그인이나 개조 없이 제품이 기본으로 갖춘 상태를 말합니다.
- ② 우회로가 있어도 안전을 보증하지 못합니다. 외부 API 를 호출하는 비공식 플러그인을 붙일 수는 있습니다. 그러나 산업용 제어에 필수적인 물리적 안전 인터록(Interlock), 쓰기 확인(Echo Check), 명령 실행 감사 로깅 같은 결정론적 안전 메커니즘을 보장할 수 없습니다.
- ③ 패널 체계가 시간 축에 고착되어 있습니다. 꺾은선 그래프, 바 차트, 게이지 표현에 맞춰 설계된 구조입니다.
- ④ P&ID 를 표현할 수 없습니다. 실제 플랜트 운전원이 필요로 하는 공정 계장 도면(P&ID), 배관 내 유체 흐름 애니메이션, 펌프·밸브의 물리적 레이아웃 기반 동적 상태 점멸(Mimic Diagram)을 엔지니어링 표준에 맞추어 시각화하는 데 심각한 제약이 따릅니다.

②의 세 단어는 처음 보면 무슨 말인지 모릅니다. 풀어 씁니다.

용어	무엇인가	없으면 무슨 일이 생기나
안전 인터록 (Interlock)	특정 조건이 되면 사람의 조작과 무관하게 강제로 설비를 정지시키는 논리. 이 강제 정지를 현장에서 트립(trip)이라고 부릅니다. 운전원이 정지 버튼을 누른 것과 달리 보호 논리가 스스로 건 정지라는 뜻이고, 조건이 풀려도 대개 사람이 리셋을 해 줘야 다시 기동됩니다. 우리 공정의 인터록은 "압력이 6.5 barg 를 넘으면 펌프를 무조건 트립"입니다.	운전원이 정지 버튼을 못 누르거나 화면이 멈춘 사이에 설비가 파손됩니다. 사람의 반응 속도에 안전을 맡기게 됩니다.
쓰기 확인 (Echo Check)	명령을 보낸 뒤 그 주소를 다시 읽어 실제로 바뀌었는지 확인하는 절차. 메아리(echo)를 되받아 보는 것입니다. 제대로 된 구현 은 되읽기까지의 제한 시간을 두고, 값이 다르면 운전원에게 실패를 알립니다.	버튼은 눌렀는데 통신이 끊겨 명령이 안 갔을 때, 화면 은 "정지"인데 펌프는 계속 도는 상태가 됩니다. 운전원이 거짓 화면을 믿게 됩니다.

용어	무엇인가	없으면 무슨 일이 생기나
명령 감사 로깅 (Command Audit Logging)	누가 언제 어느 태그에 어떤 값을 썼는지 남기는 기록.	사고 후 원인 규명이 불가능합니다. 규제 산업(제약·식품·발전)에서는 이 기록 자체가 법적 요구사항입니다.
결정론적 (Deterministic)	같은 입력이면 항상 같은 시간 안에 같은 결과가 나온다는 성질. 앞에서 본 고정 스캔 주기가 이것을 만들어 냅니다.	"보통은 0.1 초 안에 되는데 가끔 5 초 걸린다"는 시스템 에 안전을 맡길 수 없습니다.

즉 설계 보고서의 지적은 "Grafana 에 버튼이 없다"는 표면적 불평이 아니라, 플러그인으로 버튼을 억지로 만들어 도 그 버튼이 산업 제어가 요구하는 보장을 제공하지 못한다는 구조적 지적입니다.

주의 — ③·④는 지금 그대로는 맞지 않습니다. 이 교본이 보충합니다. 설계 보고서의 ③(시간 축 고착)과

④(P&ID 표현 불가)는 작성 시점의 Grafana 를 기준으로 한 서술입니다. 현재 AR-100 이 실제로 돌리는 Grafana 11.4.0 에는 Canvas 패널이 기본 패널로 들어 있습니다(.env 의 GRAFANA_VERSION=11.4.0). Canvas 패널은 요소를 격자가 아니라 좌표에 자유롭게 배치하고, SVG 아이콘을 넣고, 값에 따라 색을 바꾸고, 버튼 요소로 외부 API 를 호출할 수 있습니다. 비공식 플러그인 이 아니라 제품 기본 기능입니다. 그러므로 "Grafana 로는 도면 비슷한 화면을 아예 못 만든다"는 말은 사실이 아닙니다.

그렇다면 결론이 바뀌는가 하면, 바뀌지 않습니다. 위 표의 네 줄 — 인터록, 예코 체크, 감사 로깅, 결정론적 시간 보장 — 은 Canvas 가 있어도 그대로 없습니다. 거기에 SCADA 제품이 기본으로 갖춘 태그 체계와 알람 관리도 없습니다. 즉 반박된 것은 ④(그림을 못 그린다)이고, 살아남은 것은

①·②(제어와 안전 보장이 없다)이며, 둘을 나란히 두라는 설계 보고서의 결론은 ①·② 만으로 충분히 지탱됩니다. 논거 네 개 중 하나가 낡았다는 사실을 숨기고 넘어가면, 학생이 Canvas 를 알게 되는 순간 나머지 셋까지 의심하게 됩니다.

그래서 설계 보고서의 결론은 Grafana 를 버리라는 것이 아니라 둘을 나란히 두라는 것입니다. 원문 표현으로 "현장 설비의 실시간 상태 확인 및 양방향 기기 제어는 FUXA 또는 ThingsBoard 대시보드에 일임하고, 장기 시계열 히스토리안 트렌드 분석, 설비 종합 효율(OEE) 통계, 전체 데이터 파이프라인의 인프라 건전성 모니터링은 Grafana 가 담당하도록 이원화하는 것이 엔지니어링 관점에서 가장 이상적인 구성"입니다.

여기 나온 OEE(Overall Equipment Effectiveness, 설비 종합 효율)도 풀어 둡니다. 설비가 얼마나 잘 쓰이고 있는지를 하나의 백분율로 나타내는 제조업 표준 지표이고, 세 가지를 곱해서 만듭니다. 가동률(계획된 시간 중 실제로 돌아간 비율) × 성능(정상 속도로 났어야 할 생산량 대비 실제 생산량) × 양품률(만든 것 중 불량률이 아닌 비율). 세 값이 각각 90 % 씩이면 OEE 는 $0.9 \times 0.9 \times 0.9 = 72.9\%$ 가 됩니다. 각 항목이 괜찮아 보여도 곱하면 급격히 떨어지기 때문에, 공장에서 개선 대상을 찾을 때 널리 쓰입니다. 중요한 것은 이 지표가 지난 기간 전체를 모아야 계산된다는 점입니다. 지금 이 순간을 보는 SCADA 화면에서는 나올 수 없고, 히스토리안에 쌓인 이력에서 나옵니다. OEE 가 Grafana 쪽으로 배정된 이유가 이것입니다.

AR-100 에서는. 이 이원화를 그대로 구현했습니다. ARCHITECTURE.md 6 절이 두 화면의 경계를 못 박아 둡니다. FUXA(<http://localhost:27018>)는 실시간 감시 와 제어를 맡고 데이터원은 Modbus/TCP(제어)와 MQTT(실시간 알람)입니다. Grafana(<http://localhost:27030>)는 장기 분석 전용 읽기 화면이고 데이터원은 InfluxDB 와 Prometheus 이며, 제어 기능은 "없음"이라고 표에 명시되어 있습니다. Grafana 쪽에는 대시보드 네 장이 grafana/dashboards/ 아래 JSON 으로 들어 있습니다 — 01-process.json (AR-100 · 공정 트렌드, 패널 8 장), 02-anomaly.json (ML 이상 탐지), 03-alerts.json (알람 이력), 04-infra.json (파이프라인 인프라 건전성). 네 파일의 패널 유형을 세어 보면 timeseries 16 개, stat 2 개, table 2 개뿐입니다. Canvas 패널은 한 장도 쓰지 않았고, 어느 하나에도 배관 도면이 없습니다. 그게 이 설계에서는 정상입니다.

FUXA — 웹 브라우저에서 도는 SCADA

FUXA 의 구조와 동작 원리

FUXA 는 웹 브라우저에서 동작하는 오픈소스 SCADA/HMI 플랫폼입니다. 설계 보고서의 표현으로는 "순수 웹 브라우저 환경에서 동작하는 차세대 오픈소스 SCADA/HMI 플랫폼"입니다. 전통적인 SCADA 제품은 운전원 PC 에 전용 프로그램을 설치해야 했고, 화면 편집기도 그 PC 에서만 돌았습니다. FUXA 는 서버 하나만 띄우면 브라우저로 접속해 화면을 그리고, 같은 브라우저로 운전까지 합니다.

구조는 서버와 화면 두 부분입니다. 서버 쪽(Node.js 로 작성)은 산업 프로토콜 클라이언트를 내장합니다. 설계 보고서가 명시한 대로 Modbus, OPC-UA, MQTT 클라이언트를 자체 엔진에 탑재하고 있어, 별도의 중간 프로그램 없이 FUXA 가 직접 PLC 에 접속해 값을 읽고 값을 씁니다. 화면 쪽은 SVG(Scalable Vector Graphics, 확대해도 깨지지 않는 벡터 그림 형식)로 그려집니다. 엔지니어는 웹 GUI 에서 드래그 앤 드롭으로 산업용 SVG 심볼과 P&ID 도면을 배치하고, 모터 회전이나 배관 흐름 애니메이션을 태그에 바인딩합니다.

프로젝트 파일 하나에 전부 들어갑니다. FUXA 의 설정은 project 라는 JSON 문서 하나입니다. 우리 프로젝트는 FUXA 공식 데모 파일(project.demo.fuxap)을 내려받아 구조를 역으로 읽어 스키마를 확정했습니다. 최상위 키는 devices , hmi , version , name , charts , server 여섯 개입니다. 핵심은 앞 의 둘입니다.

devices — 디바이스 이름을 키로 하는 맵입니다. 배열이 아니라 맵이라는 점이 중요합니다. 각 디바이스 안에 접속 정보 (property)와 태그 목록 (tags)이 들어갑니다. hmi.views[] — 화면 목록입니다. 각 화면은 svgcontent 와 items 두 덩어리로 이루어집니다. hmi.views[].svgcontent — 화면 전체가 하나의 긴 SVG 문자열입니다. 도형 100 개를 그렸어도 JSON 안에서는 문자열 하나입니다. hmi.views[].items — SVG 요소의 id 를 키로 하는 바인딩 속성 맵입니다. 여기서 앞서 말한 태그 바인딩이 실제로 저장되는 자리입니다. variableId — 바인딩이 가리키는 태그의 주소 표기로, 형식은 "<디바이스명>^~^<태그 id>" 입니다. ^~^ 라는 기묘한 구분자를 씁니다. 태그 이름에 절대 나오지 않을 문자열을 골랐기 때문입니다.

동작 순서는 이렇습니다. ① FUXA 서버가 devices 의 접속 정보로 PLC 에 붙어 polling 주기마다 태그 값을 읽습니다. ② 브라우저가 화면을 열면 svgcontent 를 그대로 그림니다. ③ 서버가 웹소켓으로 최신 태그 값을 밀어 주면, 브라우저는 items 의 바인딩을 보고 어느 SVG 요소를 어떻게 바꿀지 결정합니다. 숫자 위젯이면 <text> 내용을 교체하고, 모터 위젯이면 ranges 구간에 따라 원의 색을 바꿉니다. ④ 사용자가 버튼을 누르면 그 items 항목의 이벤트 정의 ({"type": "click", "action": "onSetValue", "actparam": "1"})를 따라 서버에 쓰기 요청이 가고, 서버가 Modbus 쓰기를 실행합니다.

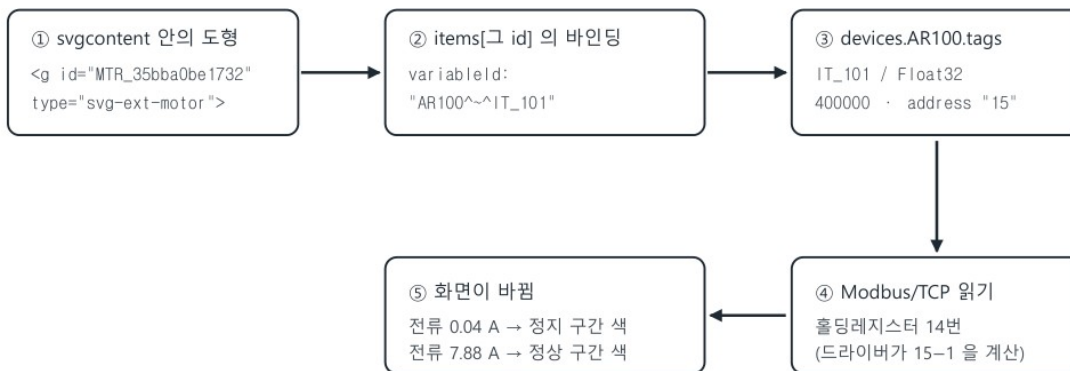
웹소켓(WebSocket)이 여기서 처음 나오니 앞에서 배운 것과 견주어 풀어 둡니다. 보통의 HTTP 는 브라우저가 한 번 묻고 서버가 한 번 답하면 연결이 끝나는 방식입니다. 그래서 값이 계속 바뀌는 화면 을 HTTP 로만 만들려면 브라우저가 1 초마다 "바뀐 거 있어요?"라고 다시 묻는 수밖에 없습니다(이것이 곧 뒤에 나올 Pull 방식입니다). 웹소켓은 다릅니다. 한 번 연결하면 그 통로를 계속 열어 두고, 서버가 아무 때나 브라우저 쪽으로 데이터를 밀어 넣을 수 있습니다. 즉 웹소켓은 서버에서 브라우저로 가는 Push 통로입니다. FUXA 가 화면을 1 초마다 새로 고치지 않고도 값이 바로바로 바뀌는 이유가 이것입니다. 정리하면 FUXA 서버는 PLC 를 향해서는 Pull(폴링)로 읽고, 브라우저를 향해서는 Push(웹소켓)로 밀어 준다는 두 겹 구조입니다.

위젯 종류는 type 속성으로 구분됩니다. 우리가 쓰는 것은 다섯 가지입니다.

type	하는 일	SVG 구조의 특징
svg-ext-value	실시간 수치 표 시	<g> 안의 <text> 내용이 태그 값으로 치환됩니다.
svg-ext-motor	회전기기 심볼	c 접두사가 붙은 요소 (cMTR_xxx)의 색이 ranges 에 따라 바뀝니다.
svg-ext-valve	제어밸브 심볼	삼각형 두 개로 이루어진 표준 밸브 기호. 개도 구간별로 색이 바뀝니다.
svg-ext-html_button	제어 버튼(쓰기)	<foreignObject> 안에 진짜 HTML <BUTTON> 을 넣습니다.
svg-ext-html_input	설정치 입력창(쓰기)	마찬가지로 <foreignObject> 안의 HTML <INPUT type="number"> .

<foreignObject> 는 SVG 그림 안에 일반 HTML 조각을 끼워 넣을 수 있게 해 주는 SVG 표준 요소입니다. SVG 에는 입력창이라는 개념이 없으므로, 버튼과 입력창만은 HTML 을 빌려 씁니다.

여기서 개도(開度)라는 말이 나왔으니 풀어 둡니다. 밸브가 얼마나 열려 있는가를 백분율로 나타낸 값 이고, 개도율도 같은 뜻으로 씁니다. 0 % 는 완전히 닫힌 상태, 100 % 는 활짝 열린 상태입니다. 우리 배출밸브 CV-101 의 기본 개도는 45 % 입니다(plant.yaml 의 valve_open_sp.default: 45). 개도를 올리면 배출 유량이 늘고 반응기 레벨이 내려갑니다.



캐그 바인딩의 연결 고리. 화면의 도형 id 하나가 Modbus 주소 하나까지 이어집니다. 이 고리 중 어느 한 칸이라도 어긋나면 값이 표시
원문 도해 · SVG 도형에서 Modbus 태그 값까지의 바인딩 (원문 272 쪽)

AR-100에서는, 화면을 손으로 그리지 않습니다. fuxa/build_project.py 가 공정 정의 파일 simulator/plant.yaml 을 읽어 fuxa/project.json 을 통째로 생성하고, fuxa/provision.py 가 그것을 POST /api/project 로 FUXA 에 주입합니다. 이렇게 설정을 사람 손 대신 파일과 API 로 자동 투입해 시스템을 곧바로 쓸 수 있는 상태로 만드는 일을 프로비저닝(provisioning)이라고 부르고, "주입"은 그중 실제로 밀어 넣는 동작을 가리킵니다. 사람이 마우스로 그린 화면은 plant.yaml 이 바뀌는 순간 어긋나기 때문에 이 방식을 택했습니다. 실제 생성 결과는 이렇습니다.

```

project.json 생성: 디바이스 2개 / Modbus 태그 20개 / 바인딩 아이템 30개 / SVG 18,044 자
아이템 구성: {'svg-ext-value': 17, 'svg-ext-motor': 2, 'svg-ext-valve': 1,
              'svg-ext-html_button': 7, 'svg-ext-html_input': 3}
  
```

이 숫자는 화면 구성과 정확히 대응합니다. 수치 표시 17 개는 계측 태그 12 개 + 설정치 되읽기 3 개 (속도·개도·온도) + 인터록 상태 1 개 + 활성 알람 1 개입니다. 모터 2 개는 펌프 P-101 과 교반기 M-101 , 밸브 1 개는 CV-101 입니다. 버튼 7 개는 펌프·교반기·히터의 기동/정지 6 개와 알람 ACK 1 개, 입력창 3 개는 세 가지 설정치입니다.

디바이스 둘 중 AR100 은 "type": "ModbusTCP" 로 plant-simulator:502 의 슬레이브 1 번에 붙습니다. Modbus 는 통신선 하나에 기기 여러 대를 붙일 수 있는 규약이라, 각 기기에 1 부터 247 까지 의 번호를 매겨 구분합니다. 이 번호가 슬레이브 ID(유닛 ID)이고, 요청 맨 앞에 붙어 "지금 묻는 상대는 몇 번인가"를 밝힙니다. 우리 시뮬레이터는 기기 한 대뿐이므로 slaveid: 1 입니다. polling 은 plant.yaml 의 scan_interval_ms: 1000 을 그대로 써서 1 초에 한 번 전체 태그를 읽습니다. 앞에서 설명한 스캔 한 바퀴가 1 초라는 뜻입니다.

둘째 디바이스 ALERTS 는 "type": "MQTTclient" 로 mqtt://emqx:1883 에 clientId fuxa-scada 로 접속해 scada/alerts/# 토픽을 QoS 1 로 구독합니다. QoS(Quality of Service, 전달 품질 등급)는 MQTT 가 메시지 전달을 어디까지 보증할지 정하는 값으로 0·1·2 셋 중 하나입니다. QoS 1 은 최소 한 번(at least once) 도착을 보증합니다. 받는 쪽이 "받았다"고 응답할 때까지 브로커가 다시 보내므로 유실은 없지만 중복은 생길 수 있는 등급입니다. 알람은 놓치면 안 되므로 0 이 아니라 1 을 썼습니다(자세한 내용은 5 장). 알람이 오는 경로는 Flink 가 만든 알람이 Kafka sensor.alerts → alert-republisher (Telegraf, 설정 파일 telegraf/alert-republish.conf) → MQTT scada/alerts/<태그> 를 거쳐 P&ID 화면의 알람 표시로 돌아오는 것입니다.

화면 아래쪽 안전 패널에는 Interlock 값 표시와 "알람 ACK" 버튼이 있고, 그 옆에 한 줄이 적혀 있습니다 — "발동 시 P-101 강제 트립. FUXA 가 아닌 현장 로직이 결정론적으로 수행". 인터록 판단은 시뮬레이터 내부(simulator/plant.py)에서 압력 6.5 barg 초과 시 트립, 5.2 barg 이하에서 리셋으로 동작합니다. 위아래 값을 다르게 둔 것은 경계에서 켜졌다 꺼졌다는 반복하지 않게 하는 히스테리시스입니다. 코드에는 한 줄이 더 있습니다 — 실제 PLC 와 똑같이 물리 진값이 아니라 직전 스캔의 트랜스미터 지시값을 참조하므로, 계기 고장으로 값이 튀어도 트립이 걸립니다. SCADA 는 그 결과를 보기만 합니다.

그래서 FUXA 는 에코 체크와 감사 로깅을 하는가

앞 표에서 Grafana 를 탈락시킨 근거가 인터록·에코 체크·감사 로깅 세 가지였으니, FUXA 가 그 셋을 제공하는지 끝까지 답해야 이 장의 논리가 닫힙니다. 정직하게 적습니다.

인터록 — 있습니다. 다만 FUXA 가 하는 일이 아닙니다. 앞에서 본 대로 시뮬레이터(= 현장 제어기 자리)가 고정 스캔 주기로 결정론적으로 수행하고, FUXA 는 Interlock 코일을 읽어 화면에 표시만 합니다. 이것이 정확히 올바른 배치입니다. SCADA 제품이 인터록을 제공하느냐가 아니라, 인터록이 SCADA 바깥에 있느냐가 맞는 질문입니다.

에코 체크 — 자동 절차는 없습니다. 눈으로 하는 확인만 있습니다. 우리 화면은 설정치 입력창 바로 옆에 그 레지스터를 되읽은 값을 나란히 띄웁니다(build_project.py 에서 number_input() 옆에 value() 를 붙여 놓은 것이 그것입니다). 속도 입력창에 85 를 넣으면 옆 숫자가 85 로 바뀌는지 운전원이 확인할 수 있습니다. FUXA 가 1 초마다 폴링해 값을 되읽기 때문에 가능한 일입니다. 그러나 이것은 표시일 뿐, 제한 시간 안에 값이 안 바뀌면 실패로 판정해 경보를 내는 절차는 아닙니다. 그 판정을 자동으로 하는 유일한 코드는 검증 스크립트 scripts/verify.py 의 t4_scada() 입니다 — 쓰기를 보낸 뒤 7~8 초를 기다렸다가 시뮬레이터 상태를 되읽어 값이 바뀌었는지 확인합니다. 즉 AR-100 의 에코 체크는 사람이 화면으로 하거나 검증 스크립트가 대신하며, 운전 중 자동 절차로는 구현되어 있지 않습니다.

명령 감사 로깅 — 없습니다. 누가 언제 어느 태그에 무엇을 썼는지 남기는 기록은 이 프로젝트에 구현되어 있지 않습니다. 애초에 FUXA 에 로그인 사용자를 나누어 두지 않았으므로 "누가"에 해당하는 정보 자체가 없습니다. 실제 규제 산업에 올리려면 이 부분을 반드시 추가해야 합니다.

다만 한 가지는 있습니다 — 데이터의 감사 추적. 7·8 장에서 본 quality 태그가 그것입니다. 실측값 (GOOD)과 보간된 추정값(INTERPOLATED_LINEAR)이 히스토리안까지 구분되어 남으므로, 사고 후에 "이 숫자는 진짜 잦 값인가, 시스템이 채워 넣은 값인가"를 되짚을 수 있습니다. 명령 쪽 감사는 없고 데이터 쪽 감사는 있는 상태입니다. 이 차이를 얼버무리면 학생이 "오픈소스로 다 된다"는 잘못된 인상을 갖게 됩니다.

Modbus 주소 — 레지스터 한 칸은 16 비트입니다

FUXA 의 데이터 타입 선택

레지스터의 16 비트 칸과 float32 두 워드 구조는 1 권 2 장에서 설명했습니다. 여기서 추가로 중요한 것은 FUXA 드라이버의 타입 이름입니다. 원문은 Float32 를 스왑 없는 ABCD, Float32MLE 를 두 워드 순서를 바꾸는 선택으로 설명합니다. AR-100 은 ABCD 로 쓰므로 Float32 를 골라야 합니다. 타입·순서가 틀려도 통신 자체는 정상으로 보이고 4.6e-41 같은 잘못된 수치가 나올 수 있습니다.

FUXA 를 붙일 때 막혔던 세 가지

주의 ① — 프로토콜 드라이버는 이미지에 들어 있지 않습니다. FUXA 는 산업 프로토콜 드라이버를 런타임 플러그인으로 설치합니다. 설치하지 않으면 디바이스가 조용히 기동되지 않습니다. 오류가 나지 않는 고장이라 진단이 가장 어렵습니다.

증상만 보면 이렇습니다. 값이 안 읽히는데 로그에 오류가 없습니다.

```
2026-09-16T09:03:09.948Z [INF] 'AR100' start
2026-09-16T09:03:09.973Z [INF] 'ALERTS' connected!    ← MQTT 는 연결됨
                                                    ← Modbus 는 연결 메시지 없음
```

여기서 단서를 어떻게 잡는가가 이 사례의 핵심입니다. 고장난 쪽만 들여다보면 아무것도 없습니다. 'AR100' start 한 줄이 전부이고, 그 줄만 보면 정상처럼 보입니다. 단서는 같은 시각에 성공한 다른 경로에 있었습니다. 디바이스가 둘이라 ALERTS (MQTT)는 25 밀리초 뒤에 connected! 를 찍었는데 AR100 (Modbus)은 끝내 찍지 않았습니다. 즉 이 프로그램은 연결에 성공하면 connected! 를 남긴다는 사실을 성공 사례가 알려 주고, 그래서 그 줄이 없다는 것 자체가 증거가 됩니다. 아무 일 도 안 일어나는 고장을 쫓을 때의 일반 방법이 이것입니다 — 정상 동작의 로그가 어떻게 생겼는지를 먼저 확보하고, 없는 줄을 찾습니다.

여기서부터는 가설을 세워 하나씩 지웁니다. 가설 1: 디바이스 타입 문자열이 틀렸다. 소스에서 확인했습니다 — 584: ModbusTCP: 'ModbusTCP', . 우리가 쓴 값과 같으므로 기각입니다. 가설 2: 드라이버 모듈이 아예 없다. 플러그인 목록을 열어 봅니다.

```
node-opcua      current=
modbus-serial   current=          ← 비어 있음 = 미설치
node-red        current=4.1.11
```

current 는 설치된 버전을 뜻하므로 비어 있으면 미설치입니다. 원인이 확정되었습니다. 설치는 API 한 번이고, provision.py 의 ensure_plugins() 가 기동 때마다 확인해 자동으로 처리하도록 넣었습니다.

```
curl -X POST http://localhost:27018/api/plugins \
-H 'Content-Type: application/json' \
-d '{"params":{"name":"modbus-serial","version":"8.0.19"}}'
```

주의 ② — FUXA 설정에 적는 주소는 1 부터 세고, 코일 영역 키는 문자열 "000000" 입니다. 이것이 이 장 전체에서 가장 잔인한 함정입니다.

```
var offset = parseInt(data.tags[id].address) - 1;
// because settings address from 1 to 65536 but communication start from 0
```

따라서 plant.yaml 에 적힌 0-based 주소에 반드시 +1 을 해서 넣어야 합니다. build_project.py 의 mb() 함수가 str(addr0 + 1) 로 처리합니다. 1-based 인 것은 Modbus 프로토콜이 아니라 FUXA 의 설정값이라는 점을 분명히 해 두십시오. 거꾸로 외우면 다른 도구에서 그대로 틀립니다.

두 번째가 더 고약합니다. Modbus 에는 네 가지 메모리 영역이 있고 FUXA 는 이를 memaddress 라는 문자열로 구분하는데, 이 문자열이 드라이버가 내부에서 만드는 키와 글자 단위로 같아야 합니다. 드라이버는 코일 영역 키를 만들 때 '000000' 이라는 리터럴을 씁니다. 리터럴(literal)이란 계산 결과가 아니라 코드에 그대로 박혀 있는 값 자체를 말합니다. 즉 코일 영역 번호 0 을 숫자로 계산해 쓰는 게 아니라, 0 이 여섯 개 적힌 문자열을 그냥 박아 넣었다는 뜻입니다.

```
const ModbusMemoryAddress = { CoilStatus: 0, DigitalInputs: 100000,
                               InputRegisters: 300000, HoldingRegisters: 400000 };
if (address < ModbusMemoryAddress.DigitalInputs) {
    if (tokenized) return formatAddress('000000', token); // ← 리터럴 '000000'
}
```

memaddress 를 "0" 으로 쓰면 키가 "0-0" 이 되는데 드라이버는 "0-000000" 을 찾습니다. 불일치의 결과는 주소 오류 메시지가 아니라 프로젝트 적재 실패입니다.

```
'AR100' load error! TypeError: Cannot read properties of undefined (reading 'Items')
```

올바른 영역 문자열은 1 권 2 장의 표와 같이 코일 "000000", 디스크리트 입력 "100000", 입력 레지스터 "300000", 홀딩 레지스터 "400000"입니다. 주소는 plant.yaml 의 0-based 값에 1 을 더합니다.

주의 ③ — 런타임 API 는 ^^ 형식을 받지 않습니다. variableId 의 "AR100^^PumpRun" 형식은 HMI 아이템 전용입니다. /api/getTagValue 와 /api/setTagValue 에는 순수 태그 id 만 넘겨야 합니다.

```
# 실패
curl "localhost:27018/api/getTagValue?ids=%5B%22AR100%5E~%5EPumpRun%22%5D"
# → "AR100^^PumpRun not found"
# 성공
curl "localhost:27018/api/getTagValue?ids=%5B%22TT_101%22%5D"
# → [{"id": "TT_101", "value": 68.32042694091797}]
```

덧붙여, device-values 웹소켓 이벤트가 빈 배열로 오는 것은 고장이 아닙니다. 서버가 값을 밀어 주는 대상은 화면을 열어 둔 브라우저인데, broadcastAll 기본값이 false 라서 뷰를 연 클라이언트가 하나도 없으면 아무것도 보내지 않습니다. 브라우저를 켜 두고 다시 보면 값이 흐릅니다.

AR-100 에서는. 생성기가 찍는 주소 확인 출력이 위 규칙들이 실제로 적용된 모습입니다. 왼쪽이 plant.yaml 의 0-based 주소, 오른쪽이 FUXA 에 들어간 1-based 값입니다.

```
주소 확인 (0-based Modbus → FUXA 1-based):
TT_101      area=400000 addr= 5 type=Float32
PT_101      area=400000 addr= 9 type=Float32
PumpRun     area=      0 addr= 1 type=Bool      (실제 저장은 "000000")
Interlock   area=      0 addr= 21 type=Bool
PumpSpeedSP area=400000 addr=101 type=Int16
```

plant.yaml 에서 TT-101 은 hr: 4 , 압력 PT-101 은 hr: 8 , 펌프 기동 코일은 addr: 0 , 인터록 상태 코일은 addr: 20 , 펌프 속도 설정치는 홀딩 레지스터 addr: 100 입니다. 전부 정확히 1 씩 커져 있습니다.

여기서 TT_101 이 5 인데 PT_101 이 9 인 것, 즉 4 차이가 나는 이유를 확인해 두십시오. 둘 사이에 TT_102 (재킷 온도)가 hr: 6 으로 끼어 있고, 각 태그가 Float32 라 두 칸씩 먹기 때문입니다. 실제 생성된 project.json 을 열어 보면 온도·레벨·압력 태그가 address 1, 3, 5, 7, 9 ... 로 2 씩 늘어섭니다. 반면 PumpRun 은 코일 1 비트짜리라 address 가 "1" , 그다음 교반기가 "2" 로 1 씩 붙습니다. 저장된 JSON 원문은 이렇습니다 —

```
{"id": "PumpRun", "type": "Bool", "memaddress": "000000", "address": "1"},
{"id": "PT_101", "type": "Float32", "memaddress": "400000", "address": "9"}.
```

양방향 제어 루프 — 실측 지금까지의 이야기가 말뿐인지 아닌지는 버튼을 눌러 물리량이 변하는지로만 판정됩니다. 검증 스크립트 scripts/verify.py 의 t4_scada() 함수가 이것을 자동으로 수행합니다. 절차는 단순합니다. ① 펌프 정지 를 쓰고 7 초 기다린 뒤 시뮬레이터 상태를 읽는다. ② 재기동과 속도 85 % 를 쓰고 8 초 기다린 뒤 다시 읽는다. ③ 정지

상태에서 유량이 0.2 미만이고 재기동 후 7.0 초과면 통과. 쓰고 나서 되읽어 확인한다는 점 에서 이것이 앞서 말한 수동 에코 체크에 해당합니다.

```
curl -X POST localhost:27018/api/setTagValue -d '{"tags":[{"id":"PumpRun","value":0}]}'
curl -X POST localhost:27018/api/setTagValue \
-d '{"tags":[{"id":"PumpRun","value":1}, {"id":"PumpSpeedSP","value":85}]}'
```

동작	FT-101 공급유량 [m ³ /h]	IT-101 펌프 전류 [A]
정상 운전	6.12	6.26
FUXA 에서 펌프 정지	0.03	0.04
FUXA 에서 재기동 + 속도 85 %	8.79	7.88

버튼이 눌린 순간 유량이 6.12 에서 0.03 으로 떨어졌고, 재기동하면서 속도 설정치를 60 %대(실측 59.9 %) 에서 85 %로 올리자 원래보다 높은 8.79 까지 올라갔습니다. 중요한 것은 이 숫자가 화면 안에서만 바뀐 값이 아니라는 점입니다. FUXA 버튼 → Modbus 쓰기 → 물리 모델 반응 → 센서값 변화 → EdgeX → EMQX → Kafka → Flink → InfluxDB → Grafana 전 구간을 돌아 나옵니다. 데모 문서 docs/DEM0.md 는 이 점을 관찰 항목 4 번으로 못 박아 두었습니다 — "Grafana 트렌드에도 3 초 내로 같은 변화가 나타남". 제어 명령이 한 바퀴를 돌아온 것입니다. 별도 측정에서 FUXA 제어 → 물리 반응은 스캔 1 회(1 초) 이내였습니다(docs/journal/09-metrics.md 9.4 지연표).

AR-100 에서는. Tier 4 검증 결과가 이렇게 나옵니다. 두 번째 항목은 FUXA 가 Modbus 로 읽은 값과 시뮬레이터의 실제 내부 값을 직접 대조한 것입니다.

Tier 4 · 웹 SCADA — 양방향 제어 (Grafana 로는 불가능한 영역)

- ✓ FUXA P&ID 자동 프로비저닝 — 디바이스 2 개, 뷰 1 개, 바인딩 아이템 30 개
- ✓ FUXA Modbus 읽기 (P&ID 실시간 표출) — TT_101=72.86 vs 시뮬레이터 72.86
- ✓ FUXA 양방향 제어 (SCADA 제어 루프) — 정지 FT-101=0.03 → 85% 재기동 8.29 m3/h

재기동 유량이 위 표의 8.79 와 이 검증 로그의 8.29 로 다릅니다. 두 실행 사이에 무엇이 달랐는지는 기록되어 있지 않습니다. 물리 모델상 같은 속도 85 %라도 흡입측 탱크 레벨과 배출 배압에 따라 유량이 달라지지만, 두 실행의 그 값들을 나란히 재 둔 기록이 없으므로 원인을 단정하지 않습니다. 확실한 것은 판정 기준이 "7.0 초과"이고 두 실행 모두 통과했다는 사실입니다. 절대값이 아니라 방향과 크기 자릿수가 판정 대상입니다. 절대값 재현이 필요한 시험이라면 초기 상태를 고정해 두고 다시 재야 합니다.

같은 자리의 다른 도구들 — 감사·제어 화면 ThingsBoard — 설계 보고서가 직접 비교한 대안입니다. 자체 웹 HMI 대시보드 빌더를 내장하고, 슬라이더·토글 스위치·제어 노브 같은 양방향 RPC(Remote Procedure Call, 원격 프로시저 호출) 위젯으로 센서와 구동기(Actuator)에 즉각적인 제어 페이로드를 전달합니다. RPC 란 다른 컴퓨터에 있는 함수를 마치 내 것처럼 불러 실행시키는 방식을 말합니다. 설계 보고서는 이 기능을 "안정적으로 제공한다"고 평가하며, FUXA 와 나란히 "현장 설비의 실시간 상태 확인 및 양방향 기기 제어를 일임할 대상"으로 꼽습니다. 본질적 차이는 제어의 표현 방식입니다. ThingsBoard 의 RPC 는 "장치에게 함수를 호출하라고 요청"하는 모델이라 기기 관리·인증·멀티테넌트에 강하지만, 화면은 위젯을 격자에 배치하는 대시보드 틀에 가깝습니다. 멀티테넌트(multi-tenant)란 서버 한 벌에 여러 고객사(테넌트)의 데이터와 사용자를 담되 서로 보이지 않게 칸막이를 치는 구조입니다. 장비를 여러 회사에 납품하고 한 서버로 관리하는 사업에 필요한 성질이지, 공장 한 곳을 운전 하는 데 필요한 성질은 아닙니다. FUXA 는 화면 전체가 하나의 SVG 도면이라 P&ID 레이아웃을 그대로 옮길 수 있습니다. AR-100 이 ThingsBoard 를 쓰지 않은 이유는 성능이 아니라 중복입니다 — 기기 관리

·Southbound 는 EdgeX 가, HMI·제어는 FUXA 가 이미 맡고 있어 역할이 100 % 겹칩니다.

Southbound(사우스바운드)는 플랫폼을 가운데 놓고 볼 때 아래쪽, 즉 현장 기기를 향하는 방향을 뜻합니다. 기기에 접속 하고 값을 읽고 명령을 내려보내는 일이 전부 Southbound 입니다. 반대로 상위 시스템·클라우드를 향하는 방향은 Northbound 라고 부릅니다. plant.yaml 의 제어 포인트 절에 제어 포인트 (Southbound) 라고 적혀 있는 것이 이 뜻입니다. Grafana — 같은 자리에 둘 수 없습니다. 위에서 본 이유(제어 없음, 안전 보장 없음) 때문이며, 대신 옆자리 (장기 트렌드·OEE·인프라 건전성)에 반드시 있어야 합니다. Node-RED 대시보드 — 흐름 기반 프로그래밍 도구에 붙은 화면입니다. 버튼으로 Modbus 쓰기를 하는 것 자체는 가능합니다. 그러나 화면이 위젯 목록으로 구성되어 P&ID 도면 표현이 어렵고, 태그 체계·알람·감사 로깅 같은 SCADA 고유 개념이 제품에 내장되어 있지 않아 전부 직접 만들어야 합니다. 흥미롭게도 FUXA 이미지 안에도 node-red 플러그인이 함께 들어 있습니다(앞의 플러그인 목록에서 current=4.1.11 로 이미 설치된 것이 그것입니다). 상용 SCADA/HMI 제품(Ignition, WinCC, AVEVA 계열 등) — 기능은 가장 넓습니다. 알람 관리, 리던던시 (이중화), 전자서명, 배치 제어, 그리고 앞에서 우리에게 없다고 확인한 명령 감사 로깅까지 제품에 포함됩니다. 대신 라이선스가 태그 수나 클라이언트 수에 비례해 붙습니다. 본 교본이 오픈소스 스택을 다루므로

채택하지 않았을 뿐, 실제 플랜트에서는 여전히 표준적인 선택지입니다. EdgeX Foundry의 Command 서비스 — PDF 비교표에서 EdgeX의 SCADA 제어 적합성은 "REST 기반 Command 마이크로서비스"로 정리됩니다. 즉 제어 명령을 받아 기기에 전달하는 통로는 제공하지만, 운전원이 보는 화면은 제공하지 않습니다. 화면과 통로는 다른 물건입니다. Apache StreamPipes — PDF 비교표에서 "분석 위주(현장 제어 기능 부재)"로 분류됩니다. 같은 자리에 올 수 없습니다.

Pull 방식 감시 시스템의 구조

Prometheus는 분산 IT 인프라, 컨테이너, 마이크로서비스의 가용성 감시와 알람에 특화된 시스템입니다. 설계 보고서의 규정이 그렇습니다. 여기서 "가용성(Availability)"은 살아 있는가, 제때 처리하고 있는가를 뜻합니다. 공정 값이 정상인지는 관심사가 아닙니다.

가장 중요한 특징은 수집 방식이 반대라는 점입니다. 우리가 8장까지 본 모든 것은 Push(밀어넣기)였습니다. 센서가 값을 만들면 브로커로 밀어 보내고, Telegraf가 InfluxDB로 밀어 넣습니다. Prometheus는 Pull(끌어오기)입니다. 감시 대상이 값을 보내는 게 아니라, Prometheus가 주기적으로 대상에게 찾아가 값을 긁어 옵니다. 이 행위를 스크랩(Scrape, 긁기)이라고 부릅니다. 설계 보고서의 표현으로 "대상 시스템의 엔드포인트(/metrics)를 정기적으로 폴링하는 주기적 HTTP Pull(Scraping) 방식을 강제"합니다.

엔드포인트(endpoint)는 프로그램이 외부에서 접속하라고 열어 둔 주소를 말합니다. 웹 주소 하나가 곧 엔드포인트 하나입니다. /metrics 엔드포인트는 감시 대상이 자기 상태를 내보이려고 열어 둔 주소이고, 여기에 HTTP로 접속하면 현재 상태가 한 줄에 하나씩 텍스트로 나옵니다. 형식은 메트릭이름 {라벨="값",...} 숫자 입니다. Prometheus가 하는 일은 놀라울 정도로 단순합니다 — 설정에 적힌 주소들을 10 초마다 한 번씩 방문해서 그 텍스트를 받아 저장합니다.

Pull 방식이 주는 이점은 세 가지입니다. ① 감시 대상은 Prometheus의 존재를 몰라도 됩니다. 그냥 /metrics 만 열어 두면 됩니다. ② 대상이 응답하지 않는 것 자체가 신호가 됩니다. Push 방식에서는 "값이 안 온다"가 고장인지 조용한 것인지 구분이 안 되지만, Pull에서는 방문했는데 응답이 없으면 명백한 장애입니다. Prometheus는 이것을 up 이라는 내장 메트릭으로 자동 기록합니다. ③ 감시 서버 하나가 대상 목록을 쥐고 있으므로 어디를 보고 있는지가 설정 파일에 그대로 드러납니다.

Pull 방식의 한계도 분명합니다. 첫째, 스크랩 주기보다 짧게 일어난 일은 볼 수 없습니다. 10 초 간격 이면 그 사이의 순간 피크는 사라집니다. 둘째, 저장 가능한 데이터 형태가 고정된 단일 Float64 수치로 극히 제한됩니다. 문자열도, 복합 구조체도, 바이너리도 저장할 수 없습니다. 장비 에러 코드나 운전 모드 문자열을 넣을 수 없다는 뜻입니다.

셋째가 센서 데이터에는 결정적인데, 수집 모델 자체가 어긋납니다. 설계 보고서는 산업 현장의 센서 데이터를 "공정 상황에 따라 밀리초 단위로 자율 푸시(Push)되는 불규칙한 이벤트 스트림"이라고 못 박습니다. 이 문장을 앞의 Pull 정의와 나란히 놓으면 충돌이 보입니다. Pull은 감시자가 정해진 시각에 찾아가 "지금 값이 뭐냐"고 묻는 모델이고, 센서 이벤트는 현장 사정에 따라 아무 때나 터져 나오는 모델입니다. 정해진 시각에 찾아가는 쪽은 사건이 언제 일어날지 모르는 대상에는 맞지 않습니다. 압력이 0.3 초 동안 튀었다가 돌아왔다면, 10 초마다 찾아가는 감시자에게 그 사건은 아예 일어나지 않은 일이 됩니다. 반대로 값이 한 시간 동안 하나도 안 변한 태그도 10 초마다 꼬박꼬박 같은 숫자를 저장하게 되어 저장 공간만 씹니다. 그래서 두 모델의 불일치는 정밀도 문제가 아니라 사건을 놓치느냐 마느냐의 문제이고, 이것이 Prometheus를 센서 저장소로 쓸 수 없는 세 번째이자 가장 근본적인 이유입니다. 뒤 에 나오는 Pull/Push 그림은 바로 이 대비를 그린 것입니다.

PromQL(Prometheus Query Language)은 저장된 값을 조회하는 전용 질의 언어입니다. SQL과 비슷한 자리를 차지하지만 문법이 전혀 다릅니다. 시계열을 다루는 함수 몇 개만 알면 대부분을 씁니다.

up == 0 — 스크랩에 실패한 대상. 가장 단순하고 가장 자주 쓰이는 표현입니다.

rate(x[5m]) — 최근 5분 구간에서 초당 증가율. 계속 커지지만 하는 누적 카운터를 "초당 몇 건"으로 바꿉니다.

increase(x[5m]) — 최근 5분 동안 총 얼마나 늘었는가. 실패 횟수처럼 절대 건수가 의미 있을 때 씁니다. sum by (라벨) (x) — 지정한 라벨별로 묶어 합산.

for 절이 알람의 품질을 결정합니다. 이것이 초보가 가장 많이 놓치는 부분입니다. 경보 규칙에 조건만 적으면, 값이 한순간만 튀어도 알람이 나갑니다. 실제 인프라에서는 순간적인 지연과 튜미가 늘 있어서 이런 알람은 하루에 수백 건씩 쏟아지고, 결국 아무도 안 보게 됩니다. for: 2m 을 붙이면 조건이 2분 내내 연속으로 참이어야 알람이 발화합니다. 그 사이 상태는 Pending(대기)이라 부르고, 조건이 중간에 풀리면 조용히 사라집니다. 알람 피로를 막는 가장 저렴하고 효과적인 장치가 이 한 줄입니다.

Alertmanager는 별도 프로그램입니다. Prometheus가 하는 일은 "규칙을 평가해 알람을 발화시키는 것"까지이고, 그 알람을 누구에게 어떻게 보낼지는 전부 Alertmanager가 맡습니다. 왜 분리했는가 하면, 감시 서버가 여러 대여도 사람에게 가는 통보 창구는 하나여야 하기 때문입니다. Alertmanager의 세 가지 핵심 기능을 풀어 씁니다.

그룹핑(Grouping) — 서버 20 대가 동시에 죽었을 때 알림 20 건을 보내는 대신 한 건으로 묶어 보냅니다. 묶는 기준은 group_by에 적습니다. group_wait는 첫 알람이 온 뒤 같은 그룹의 다른 알람을 얼마나 기다렸다가 함께 보낼지, group_interval은 그 그룹에 새 알람이 추가됐을 때 다시 보내기까지의 최소 간격, repeat_interval은 해결되지 않은 같은 알람을 다시 알릴 주기입니다. 억제(Inhibition) — 원인이 되는 알람이 떠 있는 동안 그로부터 파생된 알람을 누릅니다. 데이터베이스 서버가 통째로 죽었으면 "응답 지연", "연결 실패", "큐 적체"가 줄줄이 뜨는데, 이때 필요한 정보는 "서버가 죽었다" 한 줄뿐입니다. 규칙은 source_matchers(원인), target_matchers(억제 대상), equal(같아야 하는 라벨) 세 가지로 씁니다. 라우팅(Routing) — 알람의 라벨을 보고 어느 수신처로 보낼지 결정합니다. 심각도가 critical이면 호출기로, warning이면 채팅방으로 같은 분기를 트리 구조로 씁니다.

그리고 결정적인 제약 — 고 카디널리티(High Cardinality)에 취약합니다. 카디널리티는 서로 다른 시계열이 몇 종류나 되는지를 뜻합니다. Prometheus는 메트릭 이름 + 라벨 조합 하나당 시계열 하나를 만들고 그 인덱스를 메모리에 올립니다. 여기서 이름 하나가 시계열 하나가 아니라는 점이 핵심입니다. container_cpu_usage_seconds_total이라는 이름 하나에 컨테이너 20개 × CPU 코어별 라벨이 붙으면 시계열은 수십~수백 개가 됩니다. 라벨 조합 수가 기하급수적으로 늘면 인덱스 크기가 RAM을 초과해 프로세스가 OOM Crash(Out Of Memory, 메모리 고갈로 인한 강제 종료)를 일으킵니다. 산업 센서 데이터는 정확히 이 지뢰를 밟습니다 — 센서 시리얼 넘버, 공정 라인 번호, 설비 태그 조합이 수백만 개에 이르기 때문입니다. 그래서 설계 보고서의 결론은 단호합니다. "Prometheus를 센서 데이터 수집의 영구 저장소로 채택하는 것은 불가능하며, 두 컴포넌트는 철저하게 이원화되어 운영되어야 한다."

데는 Push가 맞습니다.

AR-100에서는 prometheus/prometheus.yml 맨 위에 설계 규약이 주석으로 박혀 있습니다 — "이 Prometheus는 공정 센서 태그를 단 하나도 저장하지 않는다." 전역 설정은 scrape_interval: 10s, evaluation_interval: 10s입니다. 버전은 prom/prometheus:v3.1.0, Alertmanager는 v0.28.0입니다.

설정에 external_labels: pipeline: ar100-iiot가 있는데, 이 라벨은 Prometheus 안에 저장되는 시계열에는 붙지 않습니다. external_labels는 이름 그대로 바깥으로 내보낼 때만 붙는 라벨이라, Alertmanager로 보내는 알람, 다른 Prometheus가 이 서버를 긁어 갈 때(페더레이션), 원격 저장소로 밀어 보낼 때(remote write)에만 붙습니다. 그러므로 Prometheus UI에서 pipeline="ar100-iiot"로 걸러 보면 아무것도 나오지 않습니다. 이 라벨의 쓸모는 알람을 받는 쪽에서 "이건 ar100-iiot 파이프라인에서 온 경보"라고 구분하는 데 있습니다.

스크랩 설정에 적합한 job_name은 6개이고, 실제 타깃은 7개입니다. flink 하나가 flink-jobmanager와 flink-taskmanager 두 주소를 갖기 때문입니다. 검증 출력의 "7/7"은 job 수가 아니라 타깃 수입니다.

job_name	타깃	무엇을 보는가
prometheus	localhost:9090	자기 자신
emqx	emqx:18083, 경로 /api/v5/prometheus/stats	세션 수, 연결 끊김, 처리량
kafka	kafka-exporter:9308	컨슈머 락 — 파이프라인 지연의 1차 지표
flink	flink-jobmanager:9249 flink-taskmanager:9249 (타깃 2개)	체크포인트 소요·실패, 백프레서, 재시작
influxdb	influxdb:8086/metrics	디스크 I/O, 쓰기 오류
cadvisor	cadvisor:8080	컨테이너 CPU·메모리

표에 나온 세 단어를 풀어 둡니다. 컨슈머 락(Consumer Lag)은 Kafka에 쌓인 마지막 메시지 번호와 소비자가 처리를 끝낸 번호의 차이(건수)입니다. 0이면 실시간으로 따라가는 중이고, 계속 커지면 처리 속도가 유입 속도를 못 따라간다는 뜻입니다. 백프레서(Backpressure)는 하류가 느려서 상류가 어쩔 수 없이 속도를 늦추게 된 상태를 말합니다. 수도관 끝을 막으면 앞쪽 압력이 올라가는 것과 같아 이런 이름이 붙었습니다. 체크포인트(Checkpoint)는 Flink가 계산 도중의 상태를 주기적으로 저장해 두는 스냅샷이고(우리 설정은 10초 간격), 장애가 나면 이 지점부터 되살아납니다. 체크포인트가 자주 실패하면 "정확히 한 번" 보장이 위협받습니다.

경보 규칙은 prometheus/rules.yml의 pipeline-health 그룹 하나뿐이며, 파일 첫 줄이 경계를 명시합니다 — "공정 이상(센서 값) 경보는 여기가 아니라 Flink → sensor.alerts → FUXA/Grafana 경로가 담당한다."

알람 이름 PromQL 식

KafkaConsumerLagHigh sum by (consumergroup, topic) (kafka_consumergroup_lag) > 5000FlinkCheckpointFailing increase(flink_jobmanager_job_numberOfFailedCheckpoints[5m]) > 2FlinkJobRestarting increase(flink_jobmanager_job_numRestarts[10m]) > 3 EMQXDisconnectSpike rate(emqx_client_disconnected[5m]) > 1TelemetryIngestStalled sum(rate(kafka_topic_partition_current_offset{topic="sensor.telemetry.raw"} [2m])) == 0 PipelineServiceDown up == 0TelemetryIngestStalled 가 for 절의 쓰임을 잘 보여줍니다. 오프셋 증가율이 2 분 내내 0 이어야 발화합니다. 순간적으로 메시지가 끊기는 것은 흔한 일이므로, 그때마다 알리면 아무도 보지 않게 됩니다.

prometheus/alertmanager.yml 은 짧습니다. group_by: [alertname, tier] 로 같은 알람 이름과 같은 계층끼리 묶고, group_wait: 10s / group_interval: 1m / repeat_interval: 1h 입니다. 수신처는 default 하나이며 파일 주석대로 데모 구성이라 로그 전용입니다. 실제 운영에서는 webhook·email·slack 수신자를 여기에 추가합니다.

주의 — 이 억제 규칙은 현재 설정에서 동작하지 않습니다. 실제 결함이니 그대로 적습니다

```
inhibit_rules:
  # 서비스 자체가 죽었으면 그 서비스에서 파생된 경보는 억제한다
  - source_matchers: [alertname = "PipelineServiceDown"]
    target_matchers: [severity = "warning"]
    equal: [tier]
```

의도는 분명합니다 — " PipelineServiceDown 이 떠 있는 동안, 같은 tier 라벨을 가진 warning 알람은 보내지 않는다." Kafka 가 죽었으면 컨슈머 락 경고는 당연한 결과이지 별개의 사건이 아니기 때문입니다.

그런데 rules.yml 의 PipelineServiceDown 에는 labels 가 { severity: critical } 하나뿐이고 tier 라벨이 없습니다. Alertmanager 의 equal 은 없는 라벨을 빈 문자열로 간주해 비교합니다. 그래서 원인 알람 쪽은 tier="", 대상 알람 쪽은 tier="2" 가 되어 영원히 같아지지 않습니다. 결과적으로 tier: "2" 를 달고 있는 KafkaConsumerLagHigh 와 EMQXDisconnectSpike 는 억제되지 않습니다. warning 알람 중 tier 가 없는 것도 없으므로, 이 억제 규칙은 현재 단 한 번도 발동하지 않습니다.

고치는 방법은 두 갈래입니다. ① equal: [tier] 를 빼서 "PipelineServiceDown 이 떠 있으면 모든 warning 을 누른다"로 넓히거나, ② 스크랩 job 별로 tier 라벨을 붙여(relabel_configs) up == 0 알람도 계층을 갖게 만드는 것입니다. ② 가 원래 의도에 맞지만 설정이 늘어납니다. 이 교본을 쓰는 시점에는 고치지 않은 상태입니다. 설정 파일을 읽고 "그렇게 되겠구나"라고 넘어가면 이런 결함은 절대 발견되지 않습니다 — 서비스를 실제로 죽여 보고 알림이 몇 건 오는지 세어 봐야 드러납니다.

역할 분리를 문서가 아니라 검사로 강제합니다 이 절은 도구 이야기가 아니라 방법 이야기입니다. 그런데 이 교본에서 가장 오래 남을 교훈일 수 있습니다.

설계 보고서 7 쪽 결론 5 는 InfluxDB 와 Prometheus 를 나누라고 요구합니다. 문제는 이런 종류의 규약이 문서로만 적어 두면 반드시 깨진다는 점입니다. 6 개월 뒤 누군가 "Grafana 에서 센서 값을 보고 싶은데 Prometheus 에 넣는 게 제일 빠르네"라고 생각하는 순간 규약은 사라지고, 그 사실은 몇 달 뒤 Prometheus 가 메모리 부족으로 죽는 날에야 드러납니다. 우리 프로젝트의 기록은 이를 한 문장으로 정리합니다 — "센서 데이터를 Prometheus 에 넣지 마세요"라는 위키 문서는 6 개월 뒤에 깨집니다.

그래서 규약을 검사 항목으로 만들었습니다. scripts/verify.py 가 Prometheus 에 등록된 메트릭 이름 목록을 받아 와서, 그 안에 공정 태그 이름이 섞여 있는지 확인합니다.

```
series = get(f"{PROM}/api/v1/label/___name___/values")["data"]
sensor_leak = [m for m in series
               if any(t.replace("-", "_").lower() in m.lower()
                      for t in ["LT_101", "TT_101", "PT_101", "pH_101"])]
check("Prometheus 에 센서 데이터 없음 (역할 분리)", not sensor_leak,
      f"메트릭 {len(series)} 중 공정태그 {len(sensor_leak)}개 — PDF p.7 결론 5")
```

AR-100 에서는. 실행 결과가 이렇습니다.

```
✓ Prometheus 에 센서 데이터 없음 (역할 분리)
  - 메트릭 993 중 공정태그 0 개 — PDF p.7 결론 5
✓ Prometheus 인프라 스크랩 — 7/7 타겟 정상:
  cadvisor, emqx, flink(x2), influxdb, kafka, prometheus
```

993 이라는 숫자를 정확히 읽어야 합니다. 이 검사가 부르는 /api/v1/label/___name___/values 는 서로 다른 "메트릭 이름"의 목록을 돌려줍니다. 즉 993 은 이름의 가짓수이지 시계열 개수가 아닙니다. 앞에서 본 대로 실제 시계열은 이름 × 라벨 조합이므로, 컨테이너 20 여 개를 감시하는 cAdvisor 하나만으로도 시계열은 수천 개가 만들어집니다. 이

프로젝트에서 시계열 총수를 따로 재 두지는 않았으므로 정확한 값은 미측정입니다. 분명한 것은 993 보다 훨씬 많다는 사실이고, 여기에 공정 태그 12 점 × 설비 × 품질 라벨까지 더해졌을 때 카디널리티가 어떻게 불어나는지는 이 기준선 위에서 가능해야 합니다. 카디널리티를 가르치는 자리에서 이름 수와 시계열 수를 섞으면 그 자체가 오개념이 됩니다.

같은 시점 InfluxDB 쪽에는 최근 5 분 기준 process 3,576 건, process_raw 3,534 건, anomaly 1,196 건, alerts 426 건이 쌓이고 있었고, quality 태그는 GOOD 15,858 건 / INTERPOLATED_LINEAR 42 건 으로 보존되어 있었습니다.

데이터는 전부 히스토리안으로, 건강 상태는 전부 감시 서버로. 경계가 실행되는 코드로 지켜집니다.

주의 — 이 검사에도 구멍이 있습니다. 코드를 다시 보십시오. 비교 대상 태그가 ["LT_101", "TT_101", "PT_101", "pH_101"] 네 개뿐입니다. 우리 공정 태그는 12 개입니다. 즉 FT-101 · IT-101 · VT-101 · CT-101 · LT-102 · TT-102 · FT-102 가 Prometheus 로 새어 들어가도 이 검사는 그대로 통과합니다. "규약을 문서가 아니라 검사로 강제한다"는 말은 맞지만, 지금 강제되고 있는 범위는 12 개 중 4 개입니다. 고치는 방법은 간단합니다 — 비교 목록을 plant.yaml 의 태그 정의에서 읽어 오게 만들면 태그가 늘어도 검사가 저절로 따라옵니다.

그리고 더 중요한 것 — 이 검사는 만들어 놓고 통과하는 것만 보면 의미가 없습니다. 반대쪽도 확인해야 합니다. 교재용 실습 과제가 바로 그것입니다. Telegraf 에 prometheus_client 출력을 추가해 일부러 센서 태그를 Prometheus 에 넣어 보고, 이 검사가 실제로 실패하는지, 그리고 태그 카디널리티가 늘 때 메모리 사용량이 어떻게 변하는지 관찰하는 과제입니다. 이때 FT-101 만 흘려 보내면 검사가 통과해 버린다는 사실도 함께 확인하게 됩니다. 깨뜨려 보지 않은 검사는 무엇을 재고 있는지 알 수 없습니다.

같은 자리의 다른 도구들 — 인프라 감시와 알림

Zabbix · Nagios · Icinga — Prometheus 이전 세대의 인프라 감시 도구들입니다. 본질적 차이는 데이터 모델입니다. 이들은 "호스트에 붙은 체크 항목"이 중심이라 서버 대수가 고정된 환경에 맞습니다. Prometheus 는 라벨이 붙은 시계열이 중심이라, 컨테이너가 수시로 생겼다 사라지는 환경에서 "이 라벨을 가진 것들 전부"를 한 줄로 질의할 수 있습니다. 우리 스택처럼 Docker 컨테이너가 20 여 개 도는 구조에는 후자가 맞습니다. VictoriaMetrics · Thanos · Cortex — Prometheus 와 경쟁이 아니라 확장 관계입니다. Prometheus 의 약점인 장기 보존과 수평 확장을 보완합니다. 질의는 PromQL 을 그대로 쓰므로 배운 것이 버려지지 않습니다. 다만 고 카디널리티 센서 데이터를 여기에 넣어도 된다는 뜻은 아닙니다. 수집 모델이 여전히 Pull 이고, 사건을 담는 일은 그대로 히스토리안의 몫입니다. Grafana 내장 Alerting — Grafana 자체에도 경보 기능이 있어 Alertmanager 없이 알람을 보낼 수 있습니다. 화면과 경보가 한곳에 있으니 편합니다. 다만 경보 규칙이 대시보드 쪽에 묶이고, 여러 감시 서버의 알람을 한 창구로 모아 그룹핑·억제하는 구조는 Alertmanager 쪽이 본업입니다. AR-100 은 파이프라인 장애 알람을 Alertmanager 로 일원화했습니다. Flink 의 알람 경로 — 같은 "알람"이라는 말을 쓰지만 완전히 다른 자리입니다. 압력이 규격을 넘었다, 교반기 전류가 오른 뒤 진동이 따라 올랐다, 오토인코더 재구성 오차가 임계를 넘었다 — 이런 공정 이상은 7 장의 Flink 가 판정해 sensor.alerts 토픽으로 내보내고, MQTT 를 거쳐 FUXA 화면과 Grafana 알람 대시보드 에 나타납니다. Prometheus/Alertmanager 는 이 경로에 관여하지 않습니다. 설비가 아픈 것과 파이프라인 이 아픈 것은 다른 사람이 다른 화면에서 봅니다.

정리 — 네 화면, 네 역할 이 장에서 다룬 것을 한 표로 접습니다. 같은 데이터에서 출발했지만 보는 사람도, 묻는 질문도, 할 수 있는 일도 다릅니다.

화면	주소	보는 사람	대답하는 질문	제어
FUXA P&ID 미믹	localhost:27018	현장 운전원	지금 이 설비가 어떤 상태인가	있음 (Modbus 직접 쓰기)
Grafana 공정·ML·알람	localhost:27030	공장 관리자 · 데이터 분석가	지난 기간 어떤 추세였고 언제 이상했나	없음
Grafana 인프라 대시보드	localhost:27030	파이프라인 운영자	수집·전송·처리가 밀리고 있나	없음
Prometheus /Alertmanager	localhost:27090 / :27093	파이프라인 운영자	어디가 죽었고 누구에게 알려야 하나	없음

그리고 이 표 어디에도 들어가지 않는 자리가 하나 더 있습니다. 현장 제어기입니다. 인터록은 이 네 화면 중 어느 것도 수행하지 않고, 화면이 전부 꺼져도 혼자 돌아갑니다. 화면은 그 결과를 읽어 보여 줄 뿐입니다. 안전은 화면 바깥에 있어야 한다는 것이 이 장에서 가장 중요한 한 줄입니다.

마지막으로 이 장의 핵심을 정리합니다. SCADA 는 화면이 예쁜 대시보드가 아니라, 화면에서 누른 버튼이 현장의 실행을 움직이고 그 결과가 다시 화면으로 돌아오는 닫힌 고리입니다. 우리는 그 고리를 유량 6.12 → 0.03 → 8.79 라는

숫자 세 개로 확인했습니다. 그리고 그 고리 밖에서, 고리 자체가 건강한지를 993 종의 메트릭 이름과 7 개 스크랩 타깃이 따로 지켜보고 있습니다. 보는 것과 움직이는 것, 지금과 과거, 설비의 건강과 파이프라인의 건강이 각자의 자리에 있습니다.

전체 연결 · 관측에서 조치와 재측정까지

원문 교본 291-294 쪽의 설명을 통한

원문의 새벽 3 시 이야기는 전체 흐름을 설명하기 위한 시나리오입니다. 55°C·10 분 지속 경보, 특정 시각의 회복, “펌프 속도를 낮춘다”와 펌프 정지 측정값의 결합은 앞 장의 실제 설정·검증을 그대로 재현한 실행 기록과 구분해야 합니다. 원문 291 쪽도 시각은 설명을 위한 것이라고 밝힙니다. 이 편집본은 그 시간표를 구현 명세로 다시 제시하지 않고, 시나리오가 설명하는 연결 관계를 아래에 남깁니다. [원문 25, 156, 291, 292 쪽]

단계	흐름·확인할 사실
관측	시뮬레이터의 지시값 → Modbus → EdgeX 내부 버스 → MQTT 반출
계약·기록	Telegraf 가 공통 계약으로 정리 → Kafka raw 에 유효 관측 보존
판정	규격·Z-Score·CEP 와 ONNX 모델이 서로 다른 기준으로 검사
저장·표시	raw/clean/score/alerts 를 구분해 InfluxDB 저장, FUXA 와 Grafana 표시
조작	사람의 화면 조작 → FUXA 의 Modbus 쓰기 → 물리 모델 반응
재측정	새로 올라온 관측으로 회복·미개선·자료 부족을 구분

명령이 전달되었다는 사실과 설비가 나아졌다는 사실은 다릅니다. 쓰기 응답·설정치 되읽기·물리 반응·일정 시간 후 재측정은 구분해야 합니다. 원문이 강조한 닫힌 고리의 핵심입니다. 동시에 9 장에서 확인한 대로 운전 중 자동 예코 체크와 명령 감사 로그는 구현되어 있지 않습니다.

고장 여섯 가지가 확인하는 층

고장	현상	확인하는 기능과 한계
spike	압력 지시값 급등	규격 경보와 현장 인터록. 실제 물리 압력과 지시값을 구분
noise	온도 흔들림 증가	롤링 Z-Score. ML 의 반응이 낮아도 역할 분담과 함께 해석
bearing_wear	전류가 오른 뒤 진동 상승	CEP 순서·시간 패턴. 개별 임계치 경보도 함께 발생 가능
drift	pH·전도도 동반 상승	단일 규격 한계는 미반응. 후속 실측에서는 Z-Score 와 ML 모두 반응
dropout	TT-101 관측 15 초 공백	raw 공백 보존, clean 에 보간값·품질 표시, 추론 입력 유지
netdown	EMQX 30 초 정지	에지 Store-and-Forward 수동 시험. 자동 검증 23 항목에는 미포함

drift 실측을 읽는 법

원문 293 쪽의 93 초 구간 비교에서 주입 전 Z-Score·ML 경보는 0 건, 주입 중 pH Z-Score 11 건·CT Z-Score 9 건·오토인코더 89 건, 종료 뒤는 모두 0 건이었습니다. 같은 표의 LT-101 규격 경보 93→93→49 건은 drift 와 무관한 기준 상태입니다. pH 는 7.385 에서 7.935 까지 올라 USL 8.4 를 넘지 않았지만 Z-Score 는 4 초 만에 반응했습니다. 2 권 7 장에 상세 비교와 측정 조건을 모았습니다. “ML 만 탐지”라는 초기 설명 대신, 단변량 변화와 다변량 재구성·기여도라는 차이를 읽어야 합니다.

숫자로 보는 전체 — 원문 측정 기록

항목	원문 값	범위·해석
전체 메모리 / CPU	9.1 GiB / 약 11%	원문 기록. Flink 3.35 GiB·Kafka 2.17 GiB 가 메모리의 약 60%
EdgeX 서비스	9 개 합계 약 150 MiB	컨테이너 수가 곧 메모리 사용량을 뜻하지는 않음
명목 센서 유입	12 건/초	12 태그×1 Hz. 별도 20 초 오프셋 차분은 252 건으로 기록
raw → clean	528 → 528	40 초 측정 구간의 처리량 비교. 모든 장애 조건의 무손실 보장은 아님
추론 시간 — 저장소 기록	0.126~0.180 ms	내장 ONNX 호출 구간. 센서부터 화면까지의 종단 지연 아님
추론 시간 — 별도 5 시간 측정	중앙값 0.178 ms / 95% 0.382 ms / 최대 15.5 ms	약 18,000 회. 최대값 원인을 GC 로 추정했으나 확인하지 않음
정상 알람	0 건 / 90 초	Z-Score 오탐 조정 후의 해당 구간
품질 분포	GOOD 99.74% / 보간 0.26%	해당 장기 누적 구간. 모든 운전의 고정 비율 아님
Prometheus	메트릭 이름 993 중 / 검사 대상 일치 0 개	전체 시계열 수 아님. 센서 누출 검사는 12 태그 중 4 개 이름만 비교
자동 검증	23 / 23 통과	구현된 검사 범위에서의 결과. netdown 과 미구현 기능은 포함하지 않음
기동 시간	최초 5~10 분 / 이후 2~3 분	환경에 따라 달라질 수 있는 원문 기록

환경: 원문은 macOS·Apple Silicon·Docker 메모리 31 GB 에서 기록된 수치라고 밝힙니다. Windows 등 다른 환경으로 그대로 일반화하지 않습니다. 원문에 등장하는 별도 실습서의 측정 기록은 이번 첨부에 포함되지 않았습니다. [원문 294 쪽]

마지막으로 확인할 구현 경계

이 자료는 한 개발 PC 에서 계층을 연결해 보여 주는 교육용 스냅샷입니다. 실제 센서의 전류 루프·ADC·교정 계층, 운영용 TLS·인증·ACL·고가용성, 자동 에코 체크·명령 감사 로그, 모델 레지스트리·무중단 갱신까지 완성한 운영 시스템으로 읽지 않습니다. 재측정으로 확인할 수 있는 것과 아직 설계 목표인 것을 나누어 읽는 것이 시스템 이해의 마지막 단계입니다.

이 권에서 통합한 반복 설명

반복 주제	편집본의 정본 위치
quality·raw/clean	2 권의 코드 정의를 반복하지 않고 저장·조회와 건수 해석에 집중
Modbus 기초	1 권 설명을 다시 출력하지 않고 FUXA 드라이버 고유 함정을 유지
인프라 지표·경보	9 장에 실제 스크랩 경로·규칙·억제 결함·검사 사각지대를 통합
전체 시나리오·수치	마지막 절에 설명용 시나리오와 측정 기록·조건을 분리

분량 감소는 반복 문장·표·설정의 통합과 연속 페이지 배치에서 만들었습니다. 원문에 없던 기능을 보충하거나, 서로 다른 측정을 임의로 하나의 정답으로 맞추지는 않았습니다. 원문 전체와 이 편집본은 설치·운영 시 함께 확인할 수 있는 별개의 자료입니다.